

Chapter 1: What Are AI Agents and Why They Matter

The Evolution: From Chat to Prompts to Agents

The journey from early chatbots to modern AI agents is one of the most important arcs in technology. Understanding this evolution is essential because it reveals why agents are not just another incremental improvement but a fundamental shift in how we use AI.

The Chat Era (2020-2022). When GPT-3 arrived, most people interacted with large language models through simple chat interfaces. You typed a message, the model replied, and that was it. Each interaction was stateless. The model had no memory of previous conversations, no ability to take actions in the world, and no capacity to reason through multi-step problems. It was a sophisticated autocomplete engine with a text box in front of it.

The Prompt Engineering Era (2022-2024). As practitioners discovered that the *way* you asked mattered enormously, prompt engineering emerged as a discipline. Techniques like few-shot prompting, chain-of-thought, and role-based prompting unlocked dramatically better results. But there was a ceiling. No matter how clever your prompt, the model was still constrained to a single input-output cycle. It could not go look something up, run code to verify its answer, or break a complex task into subtasks and execute them sequentially.

The Agent Era (2024-Present). Agents shatter that ceiling. An AI agent is not just a model responding to prompts. It is a system that can perceive its environment, reason about what to do, take actions using tools, observe the results, and continue iterating until a goal is achieved. The shift from prompting to agents is analogous to the shift from calculators to programmable computers. A calculator answers the question you type in. A computer can run a program that decides what questions to ask.

Consider a concrete example. In the prompting era, you might ask an LLM: "What are the top 5 papers on transformer efficiency published in 2025?" The model would give you its best guess based on training data, which might be outdated or fabricated. An agent given the same task would search an academic database, read abstracts, compare citation counts, filter by publication date, and return verified results with links. The difference is not intelligence. It is **agency** -- the ability to take action in pursuit of a goal.

```
# The evolution in code

# ERA 1: Simple chat
response = llm.complete("What is the capital of France?")
print(response) # "Paris"

# ERA 2: Prompt engineering
prompt = """You are an expert geographer. Think step by step.
Given the country, provide the capital along with its population
and one interesting historical fact.
Country: France"""
response = llm.complete(prompt)

# ERA 3: Agent
agent = Agent(
    llm=llm,
```

```

tools=[web_search, wikipedia_lookup, calculator],
instructions="Research the query thoroughly using available tools."
)
result = agent.run("What is the capital of France, its current population, and how
has it changed in the last decade?")
# Agent: searches web -> finds current population -> searches historical data ->
calculates change -> synthesizes answer

```

This evolution matters because each stage unlocked a new category of problems that AI could solve. Chat handled simple Q&A. Prompting handled nuanced reasoning. Agents handle complex, multi-step tasks that require interacting with the real world.

Defining AI Agents (LLM that can reason, plan, and take actions)

An **AI agent** is a software system that uses a large language model as its core reasoning engine, augmented with the ability to take actions in its environment through tools, maintain context through memory, and pursue goals through planning.

Let us break that definition into its four essential components:

- 1. Reasoning Engine (the LLM).** The language model serves as the "brain" of the agent. It interprets instructions, understands context, generates plans, and decides what to do next. The quality of the underlying LLM directly determines the agent's capability ceiling. Models like Claude, GPT-4, and Gemini provide the reasoning horsepower.
- 2. Tool Use (the hands).** Tools are functions the agent can call to interact with the outside world. A tool might search the web, read a file, execute code, send an email, or query a database. Without tools, the agent is just a chatbot with extra steps. Tools are what give agents the ability to *do* things rather than merely *say* things.
- 3. Memory (the notebook).** Agents need to remember what they have done, what they have learned, and what they are trying to accomplish. This includes short-term memory (the current conversation and task context), working memory (intermediate results and scratchpad notes), and long-term memory (persistent knowledge across sessions).
- 4. Planning (the strategy).** Agents do not just react to inputs. They formulate plans, break complex goals into subtasks, decide the order of operations, and adjust when things go wrong. Planning is what separates an agent from a simple tool-calling wrapper.

```

# The four components of an agent

class Agent:
    def __init__(self):
        self.llm = LLM("claude-3-opus")           # 1. Reasoning engine
        self.tools = ToolSet([                     # 2. Tool use
            WebSearch(),
            CodeExecutor(),
            FileManager(),
            DatabaseQuery()
        ])
        self.memory = Memory(                      # 3. Memory
            short_term=ConversationBuffer(),
            long_term=VectorStore(),

```

```

        working=Scratchpad()
    )
    self.planner = Planner(self.llm)                # 4. Planning

def run(self, goal: str):
    plan = self.planner.create_plan(goal)
    while not plan.is_complete():
        step = plan.next_step()
        result = self.execute_step(step)
        self.memory.store(step, result)
        plan.update(result)
    return plan.final_output()

```

A useful mental model: think of an agent as a capable intern with specific tools. You give the intern a goal ("research our competitors and write a summary"), and the intern figures out how to accomplish it. They might search the web, take notes, organize their findings, draft a document, review it, and revise it. They do not come back to you after every Google search asking what to do next. They exercise judgment and initiative within the bounds you have set.

What makes something an agent versus not an agent? The key differentiator is the **autonomous loop**. If the system receives an input, generates a single output, and stops, it is not an agent. If the system can decide on its own to take multiple actions, observe results, and continue working toward a goal without human intervention at each step, it is an agent. The degree of autonomy can vary, but the loop is what matters.

The Agent Loop: Observe, Think, Act, Observe

Every AI agent, regardless of framework or implementation, follows the same fundamental loop. Understanding this loop is the single most important concept in agent development.

Step 1: Observe. The agent takes in information from its environment. This includes the user's original goal, the results of any previous actions, error messages, tool outputs, and anything else relevant to the current state. All of this information forms the agent's current context.

Step 2: Think. The agent's LLM processes the current context and reasons about what to do next. This is where chain-of-thought, planning, and decision-making happen. The agent might think: "I have searched for the user's query but the results are not specific enough. I should refine my search terms and try again."

Step 3: Act. Based on its reasoning, the agent takes an action. This could be calling a tool (searching the web, running code, reading a file), generating output for the user, or deciding that the task is complete. The action produces a result.

Step 4: Observe (again). The agent observes the result of its action. Did the tool call succeed? Was the information useful? Did an error occur? This observation feeds back into Step 1, and the loop continues.

```

# The agent loop - the most fundamental pattern

def agent_loop(agent, goal: str) -> str:
    # Initialize context with the goal
    context = [{"role": "user", "content": goal}]

    max_iterations = 20 # Safety limit

```

```

for i in range(max_iterations):
    # THINK: LLM reasons about current state and decides next action
    response = agent.llm.generate(
        messages=context,
        tools=agent.tools.schemas(),
        system="You are an autonomous agent. Reason about the task, "
              "then either call a tool or provide a final answer."
    )

    # Check if agent wants to use a tool (ACT)
    if response.has_tool_call():
        tool_name = response.tool_call.name
        tool_args = response.tool_call.arguments

        # ACT: Execute the tool
        tool_result = agent.tools.execute(tool_name, tool_args)

        # OBSERVE: Add the result to context
        context.append({"role": "assistant", "content": response.text,
                       "tool_calls": [response.tool_call]})
        context.append({"role": "tool", "content": str(tool_result),
                       "tool_call_id": response.tool_call.id})
    else:
        # Agent has decided to provide a final answer
        return response.text

return "Agent reached maximum iterations without completing the task."

```

The loop is deceptively simple, but everything interesting in agent development happens within it. How does the agent decide which tool to call? That is the Think step, and it depends on prompt design, tool descriptions, and the LLM's reasoning ability. How does the agent know when it is done? That is also the Think step -- the agent must judge whether its current results satisfy the original goal.

Loop termination is one of the hardest problems in agent design. An agent can get stuck in infinite loops, repeating the same failed action, or it can terminate too early before the task is truly complete. Production agents need explicit termination conditions: maximum iteration counts, success criteria, timeout limits, and cost caps.

Agents vs. Chatbots vs. Workflows (clear comparison)

These three concepts are frequently confused. Here is a precise comparison.

Feature	Chatbot	Workflow (Chain)	Agent
Control flow	Single turn	Predetermined sequence	Dynamic, LLM-decided
Tool use	None or limited	Fixed tool sequence	Flexible tool selection
Decision making	Pattern matching	Hardcoded logic	LLM reasoning
Error handling	Generic fallback	Try/catch per step	Adaptive replanning
Autonomy	None	None	High

Predictability	High	Very high	Lower
Complexity ceiling	Low	Medium	High

A **chatbot** responds to messages. It might use an LLM to generate responses, but it does not take actions, does not plan, and does not iterate. Customer service bots that answer FAQs are chatbots. They match your question to a response pattern and reply.

A **workflow (or chain)** is a predetermined sequence of LLM calls and tool uses. "First, summarize the document. Then, extract key entities. Then, search for related articles. Then, compile a report." The steps are fixed at design time. LangChain's original "chains" concept is this pattern. Workflows are predictable and debuggable, but they cannot adapt when something unexpected happens.

An **agent** decides its own steps at runtime. Given the goal "compile a report on recent AI developments," an agent might search the web, find that one source is paywalled, try a different source, discover a relevant research paper, read it, realize it contradicts another source, investigate further, and eventually produce a report. No developer predetermined this sequence. The agent reasoned its way through it.

The practical implication: use chatbots for simple Q&A, workflows for well-understood repeatable processes, and agents for complex tasks where the path to completion is not known in advance.

Real-World Agent Examples

Understanding agents becomes much clearer when you see what they actually do in practice.

Devin (by Cognition). Devin is an AI software engineering agent. Given a task like "fix this bug in the authentication module," Devin can read the codebase, understand the architecture, identify the bug, write a fix, run tests, debug any test failures, and submit a pull request. It operates in a full development environment with a code editor, terminal, and browser. Devin represents agents at their most ambitious: autonomously performing complex professional work.

Claude Code (by Anthropic). Claude Code is an agentic coding assistant that operates in your terminal. It can read your project files, understand your codebase, write code, run tests, fix errors, and iterate. What makes it an agent rather than just a coding assistant is that it autonomously decides what files to read, what commands to run, and how to respond to errors. When a test fails, it reads the error, modifies the code, and re-runs the test without being told to.

Research Agents. Tools like Perplexity's internal systems and various open-source research agents can take a research question, search multiple sources, cross-reference information, identify contradictions, and synthesize findings into a coherent report. A human researcher might spend hours doing what a research agent can accomplish in minutes.

Customer Service Agents. Modern customer service agents go far beyond scripted chatbots. They can look up order information, process returns, troubleshoot technical issues, escalate to humans when needed, and handle complex multi-step resolutions. Companies like Intercom and Zendesk are deploying agents that handle the majority of customer inquiries without human intervention.

Data Analysis Agents. These agents can take a dataset and a question, decide what analyses to run, write and execute code to perform those analyses, interpret the results, generate visualizations, and produce a written report. They bring together code execution, data access, reasoning, and report generation in a single autonomous workflow.

The Agent Landscape: OpenAI, Anthropic, LangChain, CrewAI, AutoGen

The agent ecosystem has grown rapidly. Here is a map of the key players and frameworks.

Model Providers with Agent APIs:

- **OpenAI** provides the Assistants API, which includes built-in tool use, code interpretation, file search, and persistent threads. Their function calling interface is the most widely adopted standard for tool use. The Responses API introduced in 2025 further streamlined agent-style interactions.
- **Anthropic** offers Claude with robust tool use capabilities, extended thinking for complex reasoning, and computer use abilities. The Claude Agent SDK provides a lightweight framework for building agents directly on Claude's capabilities.
- **Google** provides Gemini models with function calling and a growing set of agent capabilities through Vertex AI Agent Builder.

Agent Frameworks:

- **LangChain / LangGraph** is the most widely adopted framework. LangChain provides abstractions for LLM interactions, while LangGraph adds stateful, graph-based orchestration for complex agent flows. It is powerful but has a significant learning curve and substantial abstraction overhead.
- **CrewAI** focuses on multi-agent collaboration. It provides a clean interface for defining agent roles, tasks, and collaboration patterns. It is particularly good for scenarios where you want specialized agents working together.
- **AutoGen (by Microsoft)** pioneered the conversational multi-agent pattern, where agents talk to each other to solve problems. It excels at research and analysis tasks.
- **Haystack (by deepset)** provides a pipeline-oriented framework that is strong for RAG-based agents and document processing workflows.
- **Semantic Kernel (by Microsoft)** offers enterprise-oriented agent capabilities with strong integration into the Microsoft ecosystem.

The build-vs-framework decision: Many experienced developers are choosing to build agents with minimal frameworks, using the model provider's API directly with custom orchestration logic. The core agent loop is simple enough that heavy frameworks can add more complexity than they remove. Start simple and add abstractions only when you need them.

Why 2024-2026 Is the Breakout Era

Several converging factors have made this specific window the moment agents went from research curiosity to practical reality.

Model capability crossed the threshold. Agents require models that can reliably follow complex instructions, reason through multi-step problems, and use tools correctly. GPT-4, Claude 3, and their successors crossed this threshold. Earlier models attempted tool use but failed too often to be practical. Current models succeed reliably enough that you can build production systems on them.

Tool use became a first-class feature. In 2023, function calling was an experimental feature. By 2025, every major model provider offers robust, well-documented tool use APIs with structured outputs, parallel tool calls, and error handling. The infrastructure caught up to the concept.

Context windows expanded dramatically. Agents need large context windows to maintain state across many steps. The jump from 4K tokens (early GPT-3.5) to 200K tokens (Claude 3) to 1M+ tokens (Gemini, Claude) means agents can work on much larger problems without losing track of what they are doing.

Cost dropped dramatically. Running an agent through 20 tool-calling iterations was prohibitively expensive in 2023. By 2025, the same workflow costs pennies. Cost reduction made agents economically viable for a wide range of applications.

The ecosystem matured. Frameworks, monitoring tools, evaluation methods, and deployment patterns have all matured. You can now build, test, deploy, and monitor agents using established tools and practices rather than inventing everything from scratch.

Enterprise adoption validated the market. Major companies began deploying agents in production, proving that they can deliver real business value. This created a virtuous cycle: more investment leads to better tools leads to wider adoption.

Key Takeaways

- AI agents are systems that use LLMs as reasoning engines, augmented with tools, memory, and planning to autonomously pursue goals.
 - The core agent loop is Observe, Think, Act, Observe -- and every agent implementation is a variation on this pattern.
 - Agents differ from chatbots (no autonomy) and workflows (no dynamic decision-making) in their ability to decide their own actions at runtime.
 - The agent landscape includes model providers (OpenAI, Anthropic, Google), frameworks (LangChain, CrewAI, AutoGen), and a growing ecosystem of tools and infrastructure.
 - The convergence of capable models, robust tool use APIs, large context windows, lower costs, and mature tooling has made 2024-2026 the breakout era for AI agents.
 - Start by understanding the fundamentals before choosing a framework. The core concepts are framework-agnostic.
-

Chapter 2: Agent Architecture Fundamentals

The ReAct Pattern: Reasoning + Acting

The **ReAct** (Reasoning + Acting) pattern is the foundational architecture for most modern AI agents. Published by Yao et al. in 2022, it formalized what many practitioners had discovered independently: LLMs perform dramatically better when they alternate between reasoning about a problem and taking actions to gather information.

Before ReAct, there were two separate approaches. **Chain-of-thought (CoT)** prompting had the model reason step by step but never interact with external tools or data. This led to plausible-sounding but sometimes factually wrong reasoning -- the model would "hallucinate" intermediate steps because it had no way to verify them. **Action-only** approaches had the model call tools in sequence but without explicit reasoning about why, leading to inefficient tool use and poor error recovery.

ReAct interleaves the two. At each step, the agent first generates a **Thought** (reasoning about the current state and what to do next), then takes an **Action** (calling a tool or producing output), then receives an **Observation** (the result of the action). This Thought-Action-Observation cycle repeats until the task is complete.

```
# The ReAct pattern in practice
```

```
REACT_SYSTEM_PROMPT = """You are a research agent. For each step, you MUST follow this format:
```

```
Thought: [Your reasoning about the current state and what to do next]
Action: [The tool to call and its parameters]
Observation: [You will receive the tool's output here]

Continue this cycle until you have enough information to answer the question.
When ready, respond with:
Thought: I now have enough information to provide a complete answer.
Final Answer: [Your complete response]"""
```

```
def react_loop(llm, tools, question: str) -> str:
    messages = [
        {"role": "system", "content": REACT_SYSTEM_PROMPT},
        {"role": "user", "content": question}
    ]

    for step in range(15):
        response = llm.generate(messages=messages, tools=tools)

        if response.contains("Final Answer:"):
            return response.extract_final_answer()

        if response.has_tool_call():
            # The "Action" step
            result = execute_tool(response.tool_call)

            # The "Observation" step -- feed result back
            messages.append({"role": "assistant", "content": response.text})
            messages.append({"role": "tool", "content": f"Observation: {result}"})
        else:
            # Model is thinking without acting -- append and continue
            messages.append({"role": "assistant", "content": response.text})

    return "Maximum steps reached."
```

Why does ReAct work so well? Three reasons. First, the explicit Thought step forces the model to plan before acting, reducing wasted tool calls. Second, the Observation step grounds the model's reasoning in real data, reducing hallucination. Third, the alternating structure creates a natural checkpoint at each step where the model can reassess and change course.

ReAct has become so foundational that most modern agent frameworks implement it by default, even if they do not use the name. When you see an agent that "reasons about its actions," you are seeing ReAct.

Core Components: LLM Brain, Tool Set, Memory, Planning Module

Every agent architecture, from the simplest to the most complex, is composed of four core components. Understanding each one in depth is essential for designing effective agents.

Component 1: The LLM Brain.

The LLM is the reasoning engine that drives everything. It interprets the user's goal, decides what tools to use, processes tool results, and generates final outputs. The choice of LLM has the single largest impact on agent

quality.

Key LLM properties that matter for agents:

- **Instruction following:** The model must reliably follow complex system prompts that define agent behavior.
- **Tool use accuracy:** The model must generate correctly formatted tool calls with appropriate parameters.
- **Reasoning depth:** The model must handle multi-step reasoning without losing track of the overall goal.
- **Context utilization:** The model must effectively use information from earlier in the conversation.
- **Calibration:** The model should know when it does not know something, rather than confabulating.

```
# LLM configuration for agent use

class AgentLLM:
    def __init__(self, model_name: str, temperature: float = 0.1):
        self.model = model_name
        # Low temperature for agents -- we want consistency, not creativity
        self.temperature = temperature
        # Higher max tokens -- agents often need room for reasoning
        self.max_tokens = 4096

    def generate(self, messages, tools=None, system_prompt=None):
        return api_call(
            model=self.model,
            messages=messages,
            tools=tools,
            temperature=self.temperature,
            max_tokens=self.max_tokens,
            system=system_prompt
        )
```

Temperature setting matters more for agents than for chat. Agents need deterministic, reliable behavior. A temperature of 0.0 to 0.2 is typical for agent reasoning. Higher temperatures introduce randomness that can cause the agent to make different decisions on the same input, making debugging nearly impossible.

Component 2: The Tool Set.

Tools are the agent's interface with the external world. Each tool is a function with a defined name, description, and parameter schema. The LLM reads these descriptions to decide which tool to call and with what arguments.

```
# Tool definition structure

class Tool:
    def __init__(self, name: str, description: str, parameters: dict, function:
callable):
        self.name = name
        self.description = description
        self.parameters = parameters # JSON Schema format
        self.function = function
```

```

def to_schema(self):
    """Convert to the format expected by the LLM API."""
    return {
        "type": "function",
        "function": {
            "name": self.name,
            "description": self.description,
            "parameters": self.parameters
        }
    }

def execute(self, **kwargs):
    """Execute the tool with given parameters."""
    try:
        result = self.function(**kwargs)
        return {"success": True, "result": result}
    except Exception as e:
        return {"success": False, "error": str(e)}

# Example: A web search tool
search_tool = Tool(
    name="web_search",
    description="Search the web for current information. Use this when you need "
        "up-to-date facts, recent events, or information not in your "
    training data.",
    parameters={
        "type": "object",
        "properties": {
            "query": {
                "type": "string",
                "description": "The search query. Be specific and use keywords."
            },
            "num_results": {
                "type": "integer",
                "description": "Number of results to return (1-10).",
                "default": 5
            }
        },
        "required": ["query"]
    },
    function=perform_web_search
)

```

Component 3: Memory.

Memory enables agents to maintain context across steps and sessions. It breaks into three categories covered in depth in Chapter 4: short-term (current conversation), working (scratchpad for the current task), and long-term (persistent storage across sessions).

Component 4: The Planning Module.

The planning module decides how to break a complex goal into steps and in what order to execute them. Simple agents do this implicitly through the LLM's reasoning. More sophisticated agents use explicit planning strategies covered in Chapter 5.

```
# Putting all four components together

class FullAgent:
    def __init__(self, model: str, tools: list, memory_config: dict):
        self.brain = AgentLLM(model)
        self.tools = ToolSet(tools)
        self.memory = MemorySystem(memory_config)
        self.planner = ReActPlanner(self.brain)

    def run(self, goal: str) -> str:
        # Store the goal in working memory
        self.memory.working.set("current_goal", goal)

        # Load relevant long-term memories
        relevant_context = self.memory.long_term.retrieve(goal, top_k=5)

        # Build the initial context
        system_prompt = self.build_system_prompt(relevant_context)

        # Run the agent loop
        result = self.planner.execute(
            goal=goal,
            tools=self.tools,
            memory=self.memory,
            system_prompt=system_prompt
        )

        # Store results in long-term memory
        self.memory.long_term.store(goal, result)

        return result
```

The Inner Monologue: How Agents Think

One of the most powerful techniques in agent design is giving the agent an explicit **inner monologue** -- a scratchpad where it can think through problems before taking action. This is distinct from chain-of-thought prompting in that it is persistent and structured.

The inner monologue serves several purposes. It helps the agent maintain focus on the overall goal across many steps. It provides a place to record observations and intermediate conclusions. It makes agent behavior explainable and debuggable -- you can read the monologue to understand why the agent made each decision.

```
# Inner monologue implementation

INNER_MONOLOGUE_PROMPT = """You are an autonomous agent. Before each action,
you MUST write your inner thoughts in a <thinking> block. Include:
```

1. GOAL REVIEW: What am I ultimately trying to accomplish?
2. PROGRESS: What have I done so far? What have I learned?
3. NEXT STEP: What should I do next and why?
4. RISKS: What could go wrong? How will I handle it?

```
<thinking>
[Your inner monologue here]
</thinking>
```

Then take your action or provide your final response."""

```
class InnerMonologueAgent:
    def __init__(self, llm, tools):
        self.llm = llm
        self.tools = tools
        self.monologue_history = []

    def step(self, messages):
        response = self.llm.generate(
            messages=messages,
            tools=self.tools,
            system=INNER_MONOLOGUE_PROMPT
        )

        # Extract and store the thinking block
        thinking = extract_thinking_block(response.text)
        if thinking:
            self.monologue_history.append({
                "step": len(self.monologue_history) + 1,
                "thought": thinking,
                "action": response.tool_call if response.has_tool_call() else
"final_answer"
            })

        return response

    def get_reasoning_trace(self) -> str:
        """Returns the full reasoning trace for debugging."""
        trace = []
        for entry in self.monologue_history:
            trace.append(f"Step {entry['step']}:")
            trace.append(f"  Thought: {entry['thought']}")
            trace.append(f"  Action: {entry['action']}")
        return "\n".join(trace)
```

Anthropic's Claude models support **extended thinking**, which provides a dedicated mechanism for this inner monologue. When enabled, the model generates reasoning tokens that are visible to the developer (for debugging) but can be structured separately from the agent's external actions. This built-in support for inner monologue makes Claude particularly well-suited for agent development.

The key insight is that **thinking is not free but it is cheap**. The tokens spent on inner monologue dramatically improve action quality and reduce wasted tool calls. An agent that spends 200 tokens thinking

before each action will typically complete tasks in fewer total steps (and fewer total tokens) than one that acts impulsively.

Single-Agent vs. Multi-Agent Architectures

The first major architectural decision is whether to use a single agent or multiple agents working together.

Single-agent architecture uses one LLM instance with one set of tools and one memory system. The agent handles all aspects of the task itself. This is simpler to build, debug, and deploy. It is the right choice for most applications.

Multi-agent architecture uses multiple LLM instances, each with potentially different roles, tools, and capabilities. Agents communicate with each other to collaboratively solve problems. This adds significant complexity but enables tackling problems that would overwhelm a single agent.

```
# Single-agent: one agent does everything
single_agent = Agent(
    llm=claude,
    tools=[search, code_exec, file_ops, database],
    instructions="You are a general-purpose assistant. Handle any task given to you."
)

# Multi-agent: specialized agents collaborate
researcher = Agent(
    llm=claude,
    tools=[web_search, paper_search, wikipedia],
    instructions="You are a researcher. Find and verify information."
)

analyst = Agent(
    llm=claude,
    tools=[code_exec, calculator, chart_generator],
    instructions="You are a data analyst. Analyze data and create visualizations."
)

writer = Agent(
    llm=claude,
    tools=[file_ops, grammar_check],
    instructions="You are a technical writer. Produce clear, well-structured reports."
)

orchestrator = Orchestrator(agents=[researcher, analyst, writer])
result = orchestrator.run("Research AI adoption trends and produce an executive report.")
```

When to use single-agent: The task can be described in one prompt. The tool set is manageable (under 15-20 tools). The task does not require fundamentally different "modes" of operation.

When to use multi-agent: The task has clearly separable subtasks requiring different expertise. You need debate or review (one agent checks another's work). The combined tool set would be too large for a single

agent to choose from effectively. You want different models for different subtasks (a fast, cheap model for simple steps and a powerful model for complex reasoning).

Stateless vs. Stateful Agents

Stateless agents treat each invocation independently. They receive a goal, execute it, return results, and forget everything. The next invocation starts from scratch.

Stateful agents maintain state across invocations. They remember previous interactions, learned preferences, accumulated knowledge, and ongoing tasks.

```
# Stateless agent -- no memory between calls
class StatelessAgent:
    def run(self, goal: str) -> str:
        messages = [{"role": "user", "content": goal}]
        return self.agent_loop(messages)

# Stateful agent -- maintains state across calls
class StatefulAgent:
    def __init__(self):
        self.conversation_history = []
        self.knowledge_base = VectorStore()
        self.user_preferences = {}
        self.task_history = []

    def run(self, goal: str) -> str:
        # Load relevant past context
        relevant_memories = self.knowledge_base.search(goal, top_k=5)

        # Build messages with history and context
        messages = self.build_context(goal, relevant_memories)

        # Run agent loop
        result = self.agent_loop(messages)

        # Store results for future reference
        self.conversation_history.append({"goal": goal, "result": result})
        self.knowledge_base.add(goal + " " + result)

        return result
```

Most production agents are stateful within a session (they remember what happened in the current task) and selectively stateful across sessions (they store important learnings but do not remember every detail). The design of what to remember and what to forget is a critical architectural decision.

The Context Window Problem and Solutions

The **context window** is the maximum amount of text an LLM can process at once. Even with today's large windows (100K to 1M+ tokens), agents routinely hit limits because each iteration adds to the context: the system prompt, conversation history, tool calls, tool results, and inner monologue all accumulate.

```
# Context management strategies

class ContextManager:
    def __init__(self, max_tokens: int = 100000):
        self.max_tokens = max_tokens
        self.reserved_tokens = 10000 # Reserved for system prompt + tools

    def manage_context(self, messages: list) -> list:
        """Keep context within limits while preserving important information."""
        total_tokens = count_tokens(messages)

        if total_tokens <= self.max_tokens:
            return messages # No management needed

        # Strategy 1: Summarize older messages
        # Keep the system prompt, first user message, and recent messages
        system = messages[0]
        first_user = messages[1]
        recent = messages[-10:]

        # Summarize everything in between
        middle = messages[2:-10]
        summary = self.summarize(middle)

        return [system, first_user,
                {"role": "system", "content": f"Summary of previous
steps:\n{summary}"},
                *recent]

    def summarize(self, messages: list) -> str:
        """Use the LLM to summarize a conversation segment."""
        summary_prompt = "Summarize the following agent steps, preserving key
findings and decisions:"
        content = "\n".join([m["content"] for m in messages])
        return llm.generate(summary_prompt + content)
```

Common solutions to context limits:

1. **Sliding window:** Drop the oldest messages as new ones arrive. Simple but loses important context.
2. **Summarization:** Periodically summarize older context into a compact form. Preserves key information but loses details.
3. **Retrieval-augmented context:** Store all context in a vector store and retrieve only what is relevant to the current step. Efficient but requires good retrieval.
4. **Hierarchical context:** Keep a running summary at one level and detailed recent context at another.
5. **External scratchpad:** Store intermediate results in a file or database that the agent can re-read when needed, rather than keeping everything in the context window.

Common Architecture Patterns

Here are the most common architecture patterns you will encounter when building agents, described in a way that makes them implementable.

Pattern 1: Simple ReAct Loop. One LLM, a set of tools, and the basic observe-think-act loop. Best for straightforward tasks with clear completion criteria. This is where you should start.

Pattern 2: Plan-and-Execute. The agent first creates a complete plan, then executes each step. A separate planning LLM call creates the plan, and a separate execution LLM call handles each step. Good for well-structured tasks.

Pattern 3: Hierarchical Agents. A high-level "manager" agent breaks the task into subtasks and delegates each to a specialized "worker" agent. The manager reviews results and coordinates. Good for complex tasks with distinct phases.

Pattern 4: Reflection Loop. After the primary agent produces a result, a "critic" agent (or the same agent in a different mode) reviews the output for quality, accuracy, and completeness. If issues are found, the primary agent revises. Good for tasks where quality matters more than speed.

Pattern 5: Human-in-the-Loop. The agent operates autonomously but pauses at certain checkpoints to request human approval or input. Good for high-stakes tasks where mistakes are costly.

```
# Pattern 2: Plan-and-Execute

class PlanAndExecuteAgent:
    def __init__(self, planner_llm, executor_llm, tools):
        self.planner = planner_llm
        self.executor = executor_llm
        self.tools = tools

    def run(self, goal: str) -> str:
        # Phase 1: Plan
        plan = self.planner.generate(
            f"Create a step-by-step plan to accomplish: {goal}\n"
            f"Available tools: {self.tools.descriptions()}\n"
            f"Output a numbered list of steps."
        )
        steps = parse_plan(plan)

        # Phase 2: Execute each step
        results = []
        for i, step in enumerate(steps):
            context = f"Overall goal: {goal}\n"
            context += f"Plan: {plan}\n"
            context += f"Previous results: {results}\n"
            context += f"Current step ({i+1}/{len(steps)}): {step}\n"
            context += "Execute this step using available tools."

            step_result = self.executor.agent_loop(context, self.tools)
            results.append({"step": step, "result": step_result})

        # Phase 3: Synthesize
        synthesis = self.executor.generate(
            f"Goal: {goal}\nAll results: {results}\n"
            f"Synthesize these results into a final answer."
```



```
)  
return synthesis
```

Key Takeaways

- The **ReAct pattern** (Reasoning + Acting) is the foundation of modern agent architecture. It interleaves thinking and doing to produce grounded, effective behavior.
- Every agent has four core components: the **LLM brain**, **tools**, **memory**, and **planning**. Understanding each is essential.
- The **inner monologue** (explicit thinking before acting) dramatically improves agent quality and debuggability.
- Choose **single-agent** architecture by default. Move to **multi-agent** only when the task genuinely requires it.
- The **context window** is a real constraint. Plan your context management strategy early.
- Start with the **simple ReAct loop** and add complexity only as needed. The fanciest architecture is the one that is appropriate for your problem, not the most sophisticated one you can build.

Chapter 3: Tool Use -- Giving Agents Superpowers

What Is Function Calling / Tool Use

Function calling (also called **tool use**) is the mechanism by which an LLM tells your application "I want to call this function with these arguments." The LLM does not actually execute the function. Instead, it produces a structured request that your code intercepts, executes, and returns the result for the LLM to process.

Here is the precise sequence:

1. You send the LLM a prompt along with a list of available tool definitions (name, description, parameter schema).
2. The LLM analyzes the prompt and decides whether it needs to use a tool. If so, it generates a structured tool call instead of (or alongside) a text response.
3. Your code receives the tool call, validates the parameters, executes the corresponding function, and captures the result.
4. You send the result back to the LLM as a tool result message.
5. The LLM processes the result and either makes another tool call or generates its final response.

```
# The complete function calling flow  
  
import anthropic  
  
client = anthropic.Anthropic()  
  
# Step 1: Define the tools  
tools = [  
    {  
        "name": "get_weather",  
        "description": "Get the current weather for a given location. Returns  
temperature, "  
        "conditions, humidity, and wind speed.",
```

```

        "input_schema": {
            "type": "object",
            "properties": {
                "location": {
                    "type": "string",
                    "description": "City and state/country, e.g. 'San Francisco,
CA'"
                },
                "units": {
                    "type": "string",
                    "enum": ["fahrenheit", "celsius"],
                    "description": "Temperature units"
                }
            },
            "required": ["location"]
        }
    }
}
]

```

Step 2: Send the request with tools

```

response = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    tools=tools,
    messages=[{"role": "user", "content": "What's the weather in Tokyo?"}]
)

```

Step 3: Check if the model wants to use a tool

```

if response.stop_reason == "tool_use":
    tool_block = next(b for b in response.content if b.type == "tool_use")
    tool_name = tool_block.name          # "get_weather"
    tool_input = tool_block.input        # {"location": "Tokyo, Japan"}
    tool_id = tool_block.id              # unique identifier

```

Step 4: Execute the tool and send the result back

```

weather_data = actual_weather_api(tool_input["location"])

```

```

follow_up = client.messages.create(
    model="claude-sonnet-4-20250514",
    max_tokens=1024,
    tools=tools,
    messages=[
        {"role": "user", "content": "What's the weather in Tokyo?"},
        {"role": "assistant", "content": response.content},
        {"role": "user", "content": [
            {
                "type": "tool_result",
                "tool_use_id": tool_id,
                "content": str(weather_data)
            }
        ]}
    ]
)

```

```
)  
# Step 5: Model processes the result and responds naturally  
print(follow_up.content[0].text)
```

The critical insight is that **the LLM decides when and how to use tools based on the tool descriptions you provide**. This means the quality of your tool descriptions directly determines how effectively the agent uses its tools. A vague description leads to incorrect usage. A clear, specific description with examples leads to reliable tool selection.

Designing Tool Interfaces

Tool design is one of the highest-leverage activities in agent development. A well-designed tool interface makes the agent more capable, more reliable, and easier to debug. A poorly designed interface causes the agent to misuse tools, call them with wrong parameters, or avoid them when they would be helpful.

Naming conventions. Tool names should be verb_noun format, descriptive, and unambiguous.

`search_web` is better than `search`. `read_file` is better than `file`. `calculate_statistics` is better than `math`. The name should make the tool's purpose obvious without reading the description.

Descriptions matter enormously. The description is the primary way the LLM understands what a tool does and when to use it. Include what the tool does, when to use it (and when not to), what it returns, and any limitations.

```
# Bad tool description  
bad_tool = {  
    "name": "search",  
    "description": "Searches for stuff",  
    "parameters": {  
        "type": "object",  
        "properties": {  
            "q": {"type": "string"}  
        }  
    }  
}  
  
# Good tool description  
good_tool = {  
    "name": "search_web",  
    "description": (  
        "Search the web for current information using a search engine. "  
        "Returns the top results with titles, URLs, and snippets. "  
        "Use this when you need up-to-date information that may not be in your  
training data, "  
        "such as recent events, current prices, or live data. "  
        "Do NOT use this for general knowledge questions you can answer directly. "  
        "Provide specific, keyword-rich queries for best results. "  
        "Returns up to 10 results; use num_results to limit if needed."  
    ),  
    "parameters": {  
        "type": "object",  
        "properties": {  
            "query": {
```

```

        "type": "string",
        "description": "Search query. Use specific keywords, not natural
language questions. "
                                "Example: 'Python 3.12 new features' instead of 'what
is new in Python'"
    },
    "num_results": {
        "type": "integer",
        "description": "Number of results to return (1-10). Default is 5.",
        "default": 5,
        "minimum": 1,
        "maximum": 10
    }
},
"required": ["query"]
}

```

Parameter design principles:

- Use **descriptive parameter names**. `query` not `q`. `file_path` not `path` (which could be ambiguous).
- Provide **type constraints**. Use enums for fixed options, min/max for numbers, patterns for strings.
- Include **defaults** for optional parameters. This reduces the chance of the LLM passing unnecessary arguments.
- Mark **required vs. optional** clearly. Only make parameters required if the tool genuinely cannot function without them.
- Add **examples** in descriptions. They help the LLM understand the expected format.

The golden rule of tool design: design tools for the LLM, not for the programmer. A programmer might be fine with a function that takes 15 parameters with subtle interactions. An LLM needs simple, focused tools with clear purposes. If a tool does too many things, split it into multiple tools.

Common Tool Categories

Agents commonly use tools in these categories:

Information Retrieval: Web search, knowledge base lookup, database queries, API calls to data sources. These tools extend the agent's knowledge beyond its training data.

Code Execution: Running Python/JavaScript/shell commands, executing SQL queries, running unit tests. These tools let the agent compute, verify, and build.

File Operations: Reading files, writing files, listing directories, searching file contents. These tools let the agent work with persistent data.

Communication: Sending emails, posting messages, creating tickets, triggering webhooks. These tools let the agent interact with people and systems.

Data Processing: Parsing documents, extracting text from PDFs, converting between formats, processing images. These tools handle data transformation.

```
# A well-organized toolset for a research agent

research_tools = [
    # Information Retrieval
    Tool("search_web", "Search the web for current information", web_search_fn),
    Tool("search_arxiv", "Search academic papers on arxiv.org", arxiv_search_fn),
    Tool("search_wikipedia", "Look up a topic on Wikipedia", wiki_fn),

    # Code Execution
    Tool("run_python", "Execute Python code and return the output", python_exec_fn),
    Tool("run_sql", "Execute a SQL query against the database", sql_exec_fn),

    # File Operations
    Tool("read_file", "Read the contents of a file", read_file_fn),
    Tool("write_file", "Write content to a file", write_file_fn),
    Tool("list_directory", "List files in a directory", list_dir_fn),

    # Data Processing
    Tool("extract_pdf_text", "Extract text from a PDF document", pdf_extract_fn),
    Tool("parse_html", "Extract main content from an HTML page", html_parse_fn),
]
```

How many tools is too many? Empirically, most LLMs handle 10-20 tools well. Beyond 30, tool selection accuracy degrades because the LLM has to reason about too many options. If you need more tools, consider organizing them hierarchically: a "meta-tool" that the agent can call to access a category of tools, or splitting across multiple specialized agents.

How Agents Choose Which Tool to Use

Understanding how LLMs select tools demystifies agent behavior and helps you design better tool interfaces.

The LLM does not "see" your Python functions. It sees the tool schemas you provide -- the names, descriptions, and parameter definitions. Based on the current conversation context and these schemas, the model uses its trained understanding of language and task decomposition to decide:

1. **Whether a tool is needed** (or if it can answer directly).
2. **Which tool** best matches the current need.
3. **What arguments** to pass.

```
# How tool selection works conceptually

# The model receives something like this in its prompt:
"""
Available tools:
1. search_web(query: str, num_results: int) - Search the web for current
information...
2. run_python(code: str) - Execute Python code and return output...
3. read_file(path: str) - Read the contents of a file...

User's question: "What is the current stock price of Apple?"
```

The model reasons:

- This requires current real-time data (stock prices change constantly)
- My training data won't have today's price
- search_web is the right tool for current information
- I should search for "Apple AAPL stock price current"

```
Tool call: search_web(query="Apple AAPL stock price current", num_results=3)
"""
```

Factors that influence tool selection:

- **Description match:** The strongest signal. If the user asks about weather and one tool says "get weather information," the match is obvious.
- **Context signals:** If previous tool results suggest a next step, the model follows the logical chain.
- **Negative descriptions:** Saying "Do NOT use this for X" actively steers the model away from incorrect usage.
- **Examples in descriptions:** Including usage examples in the description helps the model understand when the tool is appropriate.
- **Parameter compatibility:** If the model has information that matches a tool's required parameters, it is more likely to select that tool.

Building Custom Tools Step by Step

Let us build a complete custom tool from scratch. We will create a tool that lets an agent interact with a SQLite database.

```
# Step 1: Define the underlying function

import sqlite3
import json

def execute_database_query(query: str, database_path: str = "app.db") -> dict:
    """Execute a SQL query and return results."""
    try:
        conn = sqlite3.connect(database_path)
        conn.row_factory = sqlite3.Row
        cursor = conn.cursor()

        cursor.execute(query)

        if query.strip().upper().startswith("SELECT"):
            rows = cursor.fetchall()
            results = [dict(row) for row in rows]
            return {
                "success": True,
                "row_count": len(results),
                "data": results[:100] # Limit to 100 rows to avoid context overflow
            }
        else:
            conn.commit()
            return {
                "success": True,
```

```

        "rows_affected": cursor.rowcount,
        "message": f"Query executed successfully. {cursor.rowcount} rows
affected."
    }
except sqlite3.Error as e:
    return {
        "success": False,
        "error": str(e),
        "hint": "Check your SQL syntax. Common issues: missing quotes around
strings, "
                "incorrect table/column names."
    }
finally:
    conn.close()

# Step 2: Define the tool schema

database_tool_schema = {
    "name": "query_database",
    "description": (
        "Execute a SQL query against the application's SQLite database. "
        "Supports SELECT for reading data and INSERT/UPDATE/DELETE for
modifications. "
        "The database contains tables: users, orders, products, reviews. "
        "Returns results as a list of dictionaries for SELECT queries, "
        "or a confirmation message for modification queries. "
        "IMPORTANT: Always use parameterized-style queries. "
        "Limit SELECT results with LIMIT clause to avoid overwhelming context."
    ),
    "input_schema": {
        "type": "object",
        "properties": {
            "query": {
                "type": "string",
                "description": (
                    "The SQL query to execute. Examples:\n"
                    "- 'SELECT name, email FROM users WHERE active = 1 LIMIT 10'\n"
                    "- 'SELECT COUNT(*) as total FROM orders WHERE date > 2024-01-
01'\n"
                    "- 'UPDATE users SET active = 0 WHERE last_login < 2024-01-01'"
                )
            }
        },
        "required": ["query"]
    }
}

# Step 3: Create the tool handler in your agent loop

class ToolHandler:

```

```

def __init__(self):
    self.tools = {
        "query_database": execute_database_query,
    }
    self.schemas = [database_tool_schema]

def execute(self, tool_name: str, tool_input: dict) -> str:
    if tool_name not in self.tools:
        return json.dumps({"error": f"Unknown tool: {tool_name}"})

    fn = self.tools[tool_name]
    result = fn(**tool_input)
    return json.dumps(result, indent=2, default=str)

def get_schemas(self) -> list:
    return self.schemas

```

Step 4: Integrate into the agent loop

```

def agent_with_database(question: str):
    tool_handler = ToolHandler()
    messages = [{"role": "user", "content": question}]

    while True:
        response = client.messages.create(
            model="claude-sonnet-4-20250514",
            max_tokens=4096,
            tools=tool_handler.get_schemas(),
            messages=messages
        )

        if response.stop_reason == "end_turn":
            return extract_text(response)

    # Process tool calls
    for block in response.content:
        if block.type == "tool_use":
            result = tool_handler.execute(block.name, block.input)
            messages.append({"role": "assistant", "content": response.content})
            messages.append({
                "role": "user",
                "content": [{
                    "type": "tool_result",
                    "tool_use_id": block.id,
                    "content": result
                }]
            })
        break # Process one tool call at a time

```


Error Handling: What Happens When Tools Fail

Tool failures are inevitable in production agents. Network requests time out, APIs return errors, files do not exist, and databases reject malformed queries. How your agent handles these failures determines whether it is useful or frustrating.

Principle 1: Return errors as data, not exceptions. Never let a tool exception crash the agent loop. Catch all exceptions inside the tool function and return them as structured error messages that the LLM can understand and reason about.

```
# Error handling pattern

def resilient_tool_wrapper(tool_fn, tool_name: str):
    """Wraps any tool function with comprehensive error handling."""
    def wrapper(**kwargs):
        try:
            result = tool_fn(**kwargs)
            return {"status": "success", "result": result}
        except TimeoutError:
            return {
                "status": "error",
                "error_type": "timeout",
                "message": f"{tool_name} timed out. The operation took too long.",
                "suggestion": "Try a simpler query or break it into smaller parts."
            }
        except PermissionError:
            return {
                "status": "error",
                "error_type": "permission_denied",
                "message": f"Permission denied for {tool_name}.",
                "suggestion": "You may not have access to this resource."
            }
        except Exception as e:
            return {
                "status": "error",
                "error_type": type(e).__name__,
                "message": str(e),
                "suggestion": "Review the error and try a different approach."
            }
    return wrapper
```

Principle 2: Give the LLM enough information to recover. An error message of "Error" is useless. An error message of "FileNotFoundError: /data/report.csv does not exist. Available files in /data/: summary.csv, analysis.csv, report_2024.csv" lets the agent correct course.

Principle 3: Implement retry logic at the tool level. For transient errors (network timeouts, rate limits), retry automatically before returning an error to the agent. The agent should not waste reasoning tokens on "the network was slow, let me try again."

Principle 4: Set limits on consecutive errors. If a tool fails three times in a row with the same error, the agent should try a different approach rather than retrying indefinitely. Implement this in the agent loop.

```

# Agent loop with error tracking

def agent_loop_with_error_handling(agent, goal):
    consecutive_errors = 0
    max_consecutive_errors = 3
    messages = [{"role": "user", "content": goal}]

    for step in range(20):
        response = agent.llm.generate(messages=messages, tools=agent.tools)

        if response.has_tool_call():
            result = agent.execute_tool(response.tool_call)

            if result.get("status") == "error":
                consecutive_errors += 1
                if consecutive_errors >= max_consecutive_errors:
                    messages.append({
                        "role": "system",
                        "content": f"WARNING: {consecutive_errors} consecutive tool
errors. "
                                f"You must try a completely different approach."
                                })
                else:
                    consecutive_errors = 0 # Reset on success

            messages.append(tool_result_message(response, result))
        else:
            return response.text

    return "Task could not be completed."

```

Practical Example: Agent with Search, Calculator, and File Tools

Let us build a complete, working agent with three tools that can answer complex questions requiring research, calculation, and file output.

```

# Complete practical agent example

import json
import math
import os

# Tool implementations

def web_search(query: str, num_results: int = 5) -> dict:
    """Simulate web search (replace with actual API in production)."""
    # In production, this would call Google Search API, Brave Search, etc.
    search_api_url = f"https://api.search.com/v1/search?q={query}&count=
{num_results}"
    response = requests.get(search_api_url, headers={"Authorization": f"Bearer

```

```

{API_KEY}")
    results = response.json()
    return {
        "query": query,
        "results": [
            {"title": r["title"], "url": r["url"], "snippet": r["snippet"]}
            for r in results["items"][:num_results]
        ]
    }

def calculate(expression: str) -> dict:
    """Safely evaluate a mathematical expression."""
    allowed_names = {
        "abs": abs, "round": round, "min": min, "max": max,
        "sum": sum, "len": len,
        "sqrt": math.sqrt, "log": math.log, "log10": math.log10,
        "sin": math.sin, "cos": math.cos, "tan": math.tan,
        "pi": math.pi, "e": math.e, "pow": pow,
    }
    try:
        result = eval(expression, {"__builtins__": {}}, allowed_names)
        return {"expression": expression, "result": result}
    except Exception as e:
        return {"expression": expression, "error": str(e)}

def write_to_file(filename: str, content: str) -> dict:
    """Write content to a file in the output directory."""
    output_dir = "./agent_output"
    os.makedirs(output_dir, exist_ok=True)
    filepath = os.path.join(output_dir, filename)

    # Security: prevent path traversal
    if ".." in filename or filename.startswith("/"):
        return {"error": "Invalid filename. No path traversal allowed."}

    with open(filepath, "w") as f:
        f.write(content)

    return {"success": True, "filepath": filepath, "bytes_written": len(content)}

# Tool schemas
tools = [
    {
        "name": "web_search",
        "description": "Search the web for current information. Use when you need facts, "
        "statistics, or recent information not in your training data.",
        "input_schema": {
            "type": "object",
            "properties": {
                "query": {"type": "string", "description": "Search query with

```

```

specific keywords"},
    "num_results": {"type": "integer", "description": "Results to return
(1-10)", "default": 5}
    },
    "required": ["query"]
  }
},
{
  "name": "calculate",
  "description": "Evaluate a mathematical expression. Supports arithmetic, "
    "trigonometry, logarithms. Use for any calculation you need
to verify.",
  "input_schema": {
    "type": "object",
    "properties": {
      "expression": {
        "type": "string",
        "description": "Math expression, e.g. '(15000 * 0.07) / 12' or
'sqrt(144)'"
      }
    },
    "required": ["expression"]
  }
},
{
  "name": "write_to_file",
  "description": "Save content to a file. Use when asked to produce a report,
"
    "save results, or create output files.",
  "input_schema": {
    "type": "object",
    "properties": {
      "filename": {"type": "string", "description": "Name for the file,
e.g. 'report.md'"},
      "content": {"type": "string", "description": "Content to write to
the file"}
    },
    "required": ["filename", "content"]
  }
}
]

# The complete agent
def run_research_agent(question: str) -> str:
    tool_map = {
        "web_search": web_search,
        "calculate": calculate,
        "write_to_file": write_to_file,
    }

    messages = [{"role": "user", "content": question}]

```

```

system_prompt = (
    "You are a research agent that can search the web, perform calculations, "
    "and save results to files. Think step by step about how to answer the "
    "question. "
    "Use tools when needed. When your research is complete, provide a "
    "comprehensive answer."
)

for step in range(15):
    response = client.messages.create(
        model="claude-sonnet-4-20250514",
        max_tokens=4096,
        system=system_prompt,
        tools=tools,
        messages=messages
    )

    # Collect all content from the response
    assistant_content = response.content
    tool_results = []

    for block in assistant_content:
        if block.type == "tool_use":
            fn = tool_map[block.name]
            result = fn(**block.input)
            tool_results.append({
                "type": "tool_result",
                "tool_use_id": block.id,
                "content": json.dumps(result, default=str)
            })

    messages.append({"role": "assistant", "content": assistant_content})

    if tool_results:
        messages.append({"role": "user", "content": tool_results})
    else:
        # No tool calls -- agent is providing final answer
        final_text = "".join(
            block.text for block in assistant_content if hasattr(block, "text")
        )
        return final_text

    return "Agent reached step limit."

# Usage
result = run_research_agent(
    "Research the current market cap of the top 5 tech companies, "
    "calculate their combined value, and save a summary report to tech_report.md"
)
print(result)

```

This example demonstrates all the key concepts: tool definitions with clear descriptions, the agent loop with tool execution, error handling, and a practical multi-step task. The agent will search for market data, use the calculator for sums, and write the final report to a file -- all decided autonomously.

Key Takeaways

- **Function calling / tool use** is the mechanism that transforms an LLM from a text generator into an agent that can act in the world.
 - **Tool design is high-leverage work.** Clear names, detailed descriptions, typed parameters, and usage examples directly determine how effectively the agent uses its tools.
 - **Keep tool sets focused.** 10-20 well-designed tools outperform 50 poorly designed ones. Split large tool sets across specialized agents if needed.
 - **Error handling is not optional.** Tools will fail. Return errors as structured data the LLM can reason about, and implement retry logic for transient failures.
 - **Build incrementally.** Start with one or two tools, verify the agent uses them correctly, then add more. Debugging tool selection with 20 tools is much harder than with 2.
 - The LLM **chooses tools based on descriptions**, not code. Invest in writing descriptions that are clear, specific, and include usage guidance.
-

Chapter 4: Memory and Context Management

Why Agents Need Memory (the goldfish problem)

Without memory, every agent interaction starts from scratch. The agent has no idea what it did five seconds ago, what the user told it yesterday, or what it learned from previous tasks. This is the **goldfish problem** -- each moment is new, with no connection to the past.

Consider a coding agent working on a large project. Without memory, it would re-read every file from scratch each time it needs context. It would not remember that it already tried a particular approach and it failed. It would not recall the user's coding style preferences or the project's architectural decisions. Every interaction would be painfully slow and repetitive.

Memory solves four specific problems:

- 1. Continuity within a task.** An agent working on a 20-step task needs to remember what it did in steps 1-19 when executing step 20. Without short-term memory, the agent would lose track of its progress and repeat work.
- 2. Learning across tasks.** If a user tells the agent "I prefer TypeScript over JavaScript" in one session, the agent should remember this preference in the next session. Without long-term memory, users must repeat themselves constantly.
- 3. Knowledge accumulation.** A research agent that investigates a topic should be able to build on its previous findings. Without persistent memory, each research session starts from zero.
- 4. Context efficiency.** Rather than stuffing the entire history into the context window, memory systems let the agent retrieve only what is relevant to the current step. This is both cheaper and more effective.

```
# The goldfish problem illustrated

# WITHOUT memory: every call is independent
```

```

response1 = agent.run("My name is Alice and I work on the Falcon project")
# Agent knows the user is Alice working on Falcon

response2 = agent.run("What files should I modify?")
# Agent has NO IDEA who is asking or what project this is about
# It would respond: "I don't know which files you're referring to. Could you provide more context?"

# WITH memory: context persists
agent_with_memory.run("My name is Alice and I work on the Falcon project")
agent_with_memory.run("What files should I modify?")
# Agent retrieves: "User is Alice, working on Falcon project"
# It can provide a relevant, contextual answer

```

Short-Term Memory: Conversation Context

Short-term memory is the simplest form of agent memory. It is the conversation history -- the sequence of messages exchanged between the user, the agent, and any tools. Every agent has short-term memory by default because the conversation history is passed to the LLM with each request.

```

# Short-term memory is just the message list

class ShortTermMemory:
    def __init__(self, max_messages: int = 100):
        self.messages = []
        self.max_messages = max_messages

    def add_message(self, role: str, content: str):
        self.messages.append({"role": role, "content": content})
        # Trim if we exceed the limit
        if len(self.messages) > self.max_messages:
            self.trim()

    def trim(self):
        """Keep system messages, first user message, and recent messages."""
        system_msgs = [m for m in self.messages if m["role"] == "system"]
        first_user = next((m for m in self.messages if m["role"] == "user"), None)
        recent = self.messages[-20:]

        self.messages = system_msgs
        if first_user and first_user not in recent:
            self.messages.append(first_user)
        self.messages.extend(recent)

    def get_messages(self) -> list:
        return self.messages.copy()

    def get_token_count(self) -> int:
        """Estimate token count for context management."""
        total_text = " ".join(m["content"] for m in self.messages if

```

```
isinstance(m["content"], str))
    return len(total_text) // 4 # Rough estimate: 1 token per 4 characters
```

The challenge with short-term memory is that it grows linearly with the conversation. A 50-step agent interaction can easily consume 50,000+ tokens of context. You need strategies to manage this growth: trimming old messages, summarizing conversation segments, or selectively dropping tool results that are no longer relevant.

Practical tip: Tool results are often the largest items in short-term memory. A web search result might be thousands of tokens. After the agent has processed the result and drawn conclusions, the raw result is rarely needed again. Consider replacing old tool results with summaries of what the agent learned from them.

Long-Term Memory: Vector Stores, Databases, Knowledge Bases

Long-term memory persists across sessions. It lets agents remember information from previous conversations, accumulate knowledge over time, and personalize their behavior based on learned user preferences.

The most common implementation uses **vector stores** (also called vector databases). The process works like this:

1. When the agent learns something worth remembering, the information is converted into a text chunk.
2. The text chunk is converted into a numerical vector (an **embedding**) using an embedding model.
3. The vector is stored in a vector database along with the original text.
4. When the agent needs relevant context, it converts the current query into a vector and searches the database for similar vectors.
5. The most similar stored memories are retrieved and injected into the agent's context.

```
# Long-term memory with a vector store

from sentence_transformers import SentenceTransformer
import numpy as np

class LongTermMemory:
    def __init__(self, embedding_model: str = "all-MiniLM-L6-v2"):
        self.encoder = SentenceTransformer(embedding_model)
        self.memories = [] # List of {"text": str, "embedding": np.array,
                                "metadata": dict}

    def store(self, text: str, metadata: dict = None):
        """Store a new memory."""
        embedding = self.encoder.encode(text)
        self.memories.append({
            "text": text,
            "embedding": embedding,
            "metadata": metadata or {},
            "timestamp": time.time()
        })

    def retrieve(self, query: str, top_k: int = 5) -> list:
        """Retrieve the most relevant memories for a query."""
```



```

if not self.memories:
    return []

query_embedding = self.encoder.encode(query)

# Calculate cosine similarity with all stored memories
similarities = []
for memory in self.memories:
    similarity = np.dot(query_embedding, memory["embedding"]) / (
        np.linalg.norm(query_embedding) *
        np.linalg.norm(memory["embedding"])
    )
    similarities.append((similarity, memory))

# Sort by similarity and return top_k
similarities.sort(key=lambda x: x[0], reverse=True)
return [
    {"text": mem["text"], "similarity": sim, "metadata": mem["metadata"]}
    for sim, mem in similarities[:top_k]
]

def store_conversation_summary(self, conversation: list):
    """Summarize a conversation and store key points."""
    summary = llm.generate(
        "Summarize the key facts, decisions, and preferences from this conversation. "
        "Focus on information that would be useful in future conversations:\n\n"
        + format_conversation(conversation)
    )
    self.store(summary, metadata={"type": "conversation_summary"})

# Usage in an agent
class AgentWithLongTermMemory:
    def __init__(self, user_id: str):
        self.ltm = LongTermMemory()
        self.ltm.load_from_disk(f"memories/{user_id}.json") # Persist to disk

    def run(self, goal: str) -> str:
        # Retrieve relevant memories before starting
        relevant_memories = self.ltm.retrieve(goal, top_k=5)

        # Inject into system prompt
        memory_context = "\n".join([
            f"- {m['text']}" for m in relevant_memories if m['similarity'] > 0.3
        ])

        system_prompt = f"""You are a helpful agent.

Relevant information from previous interactions:
{memory_context}

Use this context to provide personalized, informed responses."""

```

```

result = self.agent_loop(goal, system_prompt)

# Store new learnings
self.ltm.store(f"User asked about: {goal}. Key outcome: {result[:200]}")
self.ltm.save_to_disk(f"memories/{self.user_id}.json")

return result

```

Production vector stores: For real applications, use a dedicated vector database rather than in-memory arrays. Popular options include **Pinecone** (managed cloud service), **Weaviate** (open source), **Chroma** (lightweight, embeddable), **pgvector** (PostgreSQL extension), and **Qdrant** (open source, performant). Each has trade-offs around scale, cost, and features.

Working Memory: Scratchpad for Multi-Step Reasoning

Working memory is the agent's scratchpad -- a structured place to store intermediate results, partial plans, and temporary state during a multi-step task. Unlike short-term memory (which is the raw conversation), working memory is organized and purpose-specific.

```

# Working memory implementation

class WorkingMemory:
    def __init__(self):
        self.scratchpad = {}          # Key-value store for named items
        self.task_state = {}          # Current task progress tracking
        self.findings = []            # Accumulated research findings
        self.decisions = []           # Decisions made and their reasoning

    def set(self, key: str, value):
        """Store a named piece of information."""
        self.scratchpad[key] = {
            "value": value,
            "timestamp": time.time()
        }

    def get(self, key: str, default=None):
        """Retrieve a named piece of information."""
        entry = self.scratchpad.get(key)
        return entry["value"] if entry else default

    def add_finding(self, finding: str, source: str = None, confidence: float = 1.0):
        """Add a research finding."""
        self.findings.append({
            "finding": finding,
            "source": source,
            "confidence": confidence,
            "timestamp": time.time()
        })

```

```

def add_decision(self, decision: str, reasoning: str):
    """Record a decision and its reasoning."""
    self.decisions.append({
        "decision": decision,
        "reasoning": reasoning,
        "timestamp": time.time()
    })

def get_summary(self) -> str:
    """Generate a summary of current working memory for the LLM."""
    parts = []

    if self.task_state:
        parts.append(f"Task State: {json.dumps(self.task_state, indent=2)}")

    if self.findings:
        parts.append("Findings:")
        for f in self.findings:
            parts.append(f" - {f['finding']} (source: {f['source']})")

    if self.decisions:
        parts.append("Decisions Made:")
        for d in self.decisions:
            parts.append(f" - {d['decision']}: {d['reasoning']}")

    if self.scratchpad:
        parts.append("Scratchpad:")
        for key, entry in self.scratchpad.items():
            parts.append(f" - {key}: {entry['value']}")

    return "\n".join(parts)

```

The key advantage of working memory over raw conversation history is structure. Instead of the LLM parsing through a long conversation to find the three data points it needs, working memory presents them in an organized, accessible format. This reduces token usage and improves reasoning accuracy.

RAG (Retrieval-Augmented Generation) for Agents

RAG combines retrieval from a knowledge base with LLM generation. For agents, RAG means the agent can access a large corpus of information without having it all in the context window.

The standard RAG pipeline for agents works as follows:

1. **Index phase:** Documents are split into chunks, embedded, and stored in a vector database.
2. **Query phase:** When the agent needs information, it formulates a search query.
3. **Retrieval phase:** The vector store returns the most relevant chunks.
4. **Generation phase:** The agent uses the retrieved chunks to inform its response or next action.

```
# RAG system for an agent
```

```
class RAGSystem:
```

```

def __init__(self, vector_store, embedding_model, chunk_size=500,
chunk_overlap=50):
    self.vector_store = vector_store
    self.embedder = embedding_model
    self.chunk_size = chunk_size
    self.chunk_overlap = chunk_overlap

def index_document(self, document: str, source: str):
    """Split a document into chunks and index them."""
    chunks = self.split_into_chunks(document)

    for i, chunk in enumerate(chunks):
        embedding = self.embedder.encode(chunk)
        self.vector_store.add(
            id=f"{source}_chunk_{i}",
            vector=embedding,
            metadata={"text": chunk, "source": source, "chunk_index": i}
        )

def split_into_chunks(self, text: str) -> list:
    """Split text into overlapping chunks."""
    words = text.split()
    chunks = []
    for i in range(0, len(words), self.chunk_size - self.chunk_overlap):
        chunk = " ".join(words[i:i + self.chunk_size])
        if chunk:
            chunks.append(chunk)
    return chunks

def query(self, question: str, top_k: int = 5) -> list:
    """Retrieve relevant chunks for a question."""
    query_embedding = self.embedder.encode(question)
    results = self.vector_store.search(query_embedding, top_k=top_k)
    return [
        {"text": r.metadata["text"], "source": r.metadata["source"], "score":
r.score}
        for r in results
    ]

# RAG as a tool for the agent
def create_rag_tool(rag_system: RAGSystem) -> dict:
    """Create an agent tool that queries the RAG system."""

def search_knowledge_base(query: str, num_results: int = 5) -> dict:
    results = rag_system.query(query, top_k=num_results)
    return {
        "query": query,
        "results": results,
        "note": "These are excerpts from the knowledge base. "
        "Use them to inform your response."
    }

```

```

return {
  "function": search_knowledge_base,
  "schema": {
    "name": "search_knowledge_base",
    "description": "Search the company knowledge base for relevant
information. "
                "Contains product documentation, internal policies, "
                "technical specifications, and historical records.",
    "input_schema": {
      "type": "object",
      "properties": {
        "query": {
          "type": "string",
          "description": "Search query. Be specific. Use keywords from
the domain."
        },
        "num_results": {
          "type": "integer",
          "description": "Number of document chunks to retrieve (1-
10).",
          "default": 5
        }
      },
      "required": ["query"]
    }
  }
}

```

RAG for agents differs from RAG for chat in important ways. In chat, the user provides the query and the system retrieves once. In agents, the agent itself formulates queries dynamically based on its current needs, and it may retrieve multiple times during a single task. The agent might search for "authentication architecture," read the results, realize it also needs information about "rate limiting," and search again. This iterative, agent-driven retrieval is sometimes called **agentic RAG**.

Memory Architecture Patterns

Different tasks require different memory configurations. Here are the most common patterns and when to use each.

Pattern 1: Conversation Buffer Only. Store the raw conversation history. No long-term memory, no structured working memory. Best for simple, single-session tasks that complete in a few steps.

Pattern 2: Conversation Buffer + Summarization. Periodically summarize older conversation history to keep the context manageable. Good for longer single-session tasks (20+ steps) where you need to manage context window limits.

Pattern 3: Conversation + RAG. Combine conversation history with retrieval from a knowledge base. Good for agents that need domain-specific knowledge (customer support agents, documentation assistants).

Pattern 4: Full Memory Stack. Short-term (conversation), working (structured scratchpad), and long-term (vector store with cross-session persistence). Good for sophisticated agents that operate over multiple sessions and need to learn from experience.

Pattern	Complexity	Use Case	Context Cost
Buffer Only	Low	Simple single-turn tasks	Grows linearly
Buffer + Summary	Medium	Long multi-step tasks	Controlled
Buffer + RAG	Medium	Knowledge-intensive tasks	Moderate
Full Memory Stack	High	Persistent, learning agents	Optimized

Implementation Patterns with Code Examples

Let us implement a complete memory system that combines all three types.

```
# Complete memory system implementation

class AgentMemorySystem:
    """Unified memory system combining short-term, working, and long-term memory."""

    def __init__(self, user_id: str, vector_store=None):
        # Short-term: conversation history
        self.conversation = []
        self.max_conversation_tokens = 50000

        # Working: structured scratchpad for current task
        self.working = WorkingMemory()

        # Long-term: persistent vector store
        self.long_term = LongTermMemory()
        if vector_store:
            self.long_term.vector_store = vector_store

        self.user_id = user_id

    def build_context(self, current_goal: str) -> list:
        """Build the complete context for the LLM from all memory sources."""
        messages = []

        # 1. System prompt with long-term memory context
        relevant_memories = self.long_term.retrieve(current_goal, top_k=5)
        memory_section = ""
        if relevant_memories:
            memory_section = "\n\nRelevant information from previous sessions:\n"
            for mem in relevant_memories:
                if mem["similarity"] > 0.3: # Only include if reasonably relevant
                    memory_section += f"- {mem['text']}\n"

        # 2. Working memory summary
        working_summary = self.working.get_summary()
        working_section = ""
        if working_summary:
            working_section = f"\n\nCurrent task state:\n{working_summary}"
```

```

system_content = (
    "You are an autonomous agent."
    + memory_section
    + working_section
)
messages.append({"role": "system", "content": system_content})

# 3. Conversation history (short-term memory)
# Apply summarization if needed
conversation = self.get_managed_conversation()
messages.extend(conversation)

return messages

def get_managed_conversation(self) -> list:
    """Return conversation history, summarizing if too long."""
    total_tokens = sum(
        estimate_tokens(m.get("content", "")) for m in self.conversation
    )

    if total_tokens <= self.max_conversation_tokens:
        return self.conversation.copy()

    # Summarize older messages, keep recent ones
    split_point = len(self.conversation) // 2
    older = self.conversation[:split_point]
    recent = self.conversation[split_point:]

    summary = self.summarize_messages(older)

    return [
        {"role": "system", "content": f"Summary of earlier conversation:\n{summary}"},
        *recent
    ]

def summarize_messages(self, messages: list) -> str:
    """Summarize a list of messages into a concise summary."""
    formatted = "\n".join([
        f"{m['role']}: {m.get('content', '[tool call]')}"
        for m in messages
    ])
    return llm.generate(
        f"Summarize this conversation segment concisely. "
        f"Preserve key facts, decisions, and results:\n\n{formatted}"
    )

def add_message(self, role: str, content):
    """Add a message to conversation history."""
    self.conversation.append({"role": role, "content": content})

```

```

def end_session(self):
    """Called when a session ends. Stores important info in long-term memory."""
    if self.conversation:
        # Store a summary of the session
        session_summary = self.summarize_messages(self.conversation)
        self.long_term.store(
            session_summary,
            metadata={"type": "session_summary", "user_id": self.user_id}
        )

        # Store any findings from working memory
        for finding in self.working.findings:
            self.long_term.store(
                finding["finding"],
                metadata={"type": "finding", "source": finding.get("source")}
            )

        # Persist to disk
        self.long_term.save_to_disk(f"memories/{self.user_id}.json")

```

This implementation shows how the three memory types work together. Short-term memory handles the current conversation. Working memory provides structured storage for the current task. Long-term memory persists across sessions and retrieves relevant context via vector similarity search.

Key Takeaways

- **Memory is essential** for agents to maintain continuity, learn from experience, and operate efficiently. Without memory, every interaction starts from scratch.
- **Short-term memory** (conversation history) is the baseline. Every agent needs it, but it grows linearly and must be managed.
- **Long-term memory** (vector stores) enables cross-session persistence. Use embedding-based retrieval to find relevant past information.
- **Working memory** (structured scratchpad) keeps the current task organized and reduces the LLM's cognitive burden.
- **RAG** is a specific application of long-term memory that lets agents access large knowledge bases efficiently.
- **Choose the right memory pattern** for your use case. Start simple (conversation buffer) and add complexity only when needed.
- Memory management is a **continuous design challenge**, not a one-time decision. Plan for context growth and implement summarization and retrieval strategies early.

Chapter 5: Planning and Multi-Step Reasoning

How Agents Break Down Complex Tasks

Complex tasks are the entire reason agents exist. A simple question like "What is the capital of France?" does not need an agent. But "Research the regulatory landscape for AI in the EU, compare it with the US approach, identify key differences, and produce a briefing document for our legal team" requires breaking the problem into manageable pieces, executing each piece, and synthesizing the results.

Agents decompose complex tasks through a process analogous to how humans tackle big projects. We instinctively break "write a research paper" into "choose a topic, do a literature review, outline the paper, write each section, revise, format, submit." Agents do the same thing, but the decomposition happens through the LLM's reasoning.

```
# Task decomposition in action

DECOMPOSITION_PROMPT = """Given a complex task, break it into a sequence of
concrete,
executable steps. Each step should be specific enough that it could be accomplished
with available tools in a single agent iteration.

Available tools: {tool_descriptions}

Task: {task}

Output a numbered list of steps. For each step, indicate:
1. What to do
2. Which tool(s) might be needed
3. What the expected output is
4. What this step depends on (which previous steps must complete first)"""

def decompose_task(llm, task: str, tools: list) -> list:
    tool_descriptions = "\n".join([f"- {t.name}: {t.description}" for t in tools])

    response = llm.generate(
        DECOMPOSITION_PROMPT.format(
            tool_descriptions=tool_descriptions,
            task=task
        )
    )

    steps = parse_numbered_list(response)
    return steps

# Example output for "Research AI regulations in EU vs US":
# 1. Search for current EU AI Act provisions and key requirements
#   Tools: web_search
#   Output: Summary of EU AI Act key points
#   Dependencies: None
#
# 2. Search for current US AI regulatory framework
#   Tools: web_search
#   Output: Summary of US AI regulations
#   Dependencies: None
#
# 3. Search for expert comparisons of EU vs US approaches
#   Tools: web_search
#   Output: Expert analysis and comparison points
#   Dependencies: None
#
```

```
# 4. Analyze and compare the two frameworks
#   Tools: none (LLM reasoning)
#   Output: Structured comparison with key differences
#   Dependencies: Steps 1, 2, 3
#
# 5. Write briefing document
#   Tools: write_to_file
#   Output: Formatted briefing document
#   Dependencies: Step 4
```

The quality of task decomposition depends heavily on the LLM's understanding of the available tools and the domain. An agent with web search and file writing tools will decompose differently than one with database access and code execution. The tool set shapes the plan.

Planning Strategies: Top-Down, Iterative, Hierarchical

Top-Down Planning creates a complete plan upfront before executing any steps. The agent analyzes the goal, breaks it into all necessary steps, orders them, and then executes the plan sequentially.

Advantages: Clear structure, predictable execution, easy to present the plan to the user for approval.

Disadvantages: Rigid -- if an early step produces unexpected results, the entire plan may need revision.

```
# Top-down planning

class TopDownPlanner:
    def __init__(self, llm, tools):
        self.llm = llm
        self.tools = tools

    def plan(self, goal: str) -> list:
        """Create a complete plan upfront."""
        response = self.llm.generate(
            f"Create a complete, detailed plan to accomplish: {goal}\n"
            f"Available tools: {self.tools.descriptions()}\n"
            f"List ALL steps needed from start to finish."
        )
        return parse_plan(response)

    def execute(self, goal: str) -> str:
        plan = self.plan(goal)
        results = []

        for step in plan:
            result = self.execute_step(step, results)
            results.append(result)

        return self.synthesize(goal, results)
```

Iterative Planning does not create a complete plan upfront. Instead, the agent plans one or a few steps ahead, executes them, observes the results, and then plans the next steps. This is essentially the ReAct pattern applied to planning.

Advantages: Highly adaptive -- the plan evolves based on what the agent learns. Disadvantages: Can wander, may miss the big picture, harder to estimate completion time.

```
# Iterative planning

class IterativePlanner:
    def __init__(self, llm, tools):
        self.llm = llm
        self.tools = tools

    def run(self, goal: str) -> str:
        history = []

        for iteration in range(20):
            # Plan the next step based on current state
            next_action = self.llm.generate(
                f"Goal: {goal}\n"
                f"What you have done so far: {history}\n"
                f"Available tools: {self.tools.descriptions()}\n"
                f"What is the single best next step? Or, if the goal is achieved, "
                f"say DONE and provide the final answer."
            )

            if "DONE" in next_action:
                return extract_final_answer(next_action)

            result = self.execute_action(next_action)
            history.append({"action": next_action, "result": result})

        return "Maximum iterations reached."
```

Hierarchical Planning combines both approaches. A high-level plan breaks the goal into major phases. Within each phase, iterative planning handles the details. This provides structure at the macro level and flexibility at the micro level.

```
# Hierarchical planning

class HierarchicalPlanner:
    def __init__(self, llm, tools):
        self.llm = llm
        self.tools = tools

    def run(self, goal: str) -> str:
        # Level 1: High-level plan (major phases)
        phases = self.llm.generate(
            f"Break this goal into 3-5 major phases: {goal}\n"
            f"Each phase should be a distinct stage of work."
        )
        phases = parse_phases(phases)

        phase_results = []
```

```

    for phase in phases:
        # Level 2: Detailed planning within each phase (iterative)
        phase_result = self.execute_phase(phase, phase_results)
        phase_results.append(phase_result)

    return self.synthesize(goal, phase_results)

def execute_phase(self, phase: str, previous_results: list) -> str:
    """Execute a single phase using iterative planning."""
    planner = IterativePlanner(self.llm, self.tools)
    context = f"Phase: {phase}\nPrevious phase results: {previous_results}"
    return planner.run(context)

```

Chain-of-Thought in Agents vs. Simple Prompting

Chain-of-thought (CoT) in standalone prompting and CoT in agents serve similar purposes but operate differently.

In **simple prompting**, CoT means adding "Let's think step by step" or providing reasoning examples in the prompt. The model reasons through the problem in a single generation. All reasoning happens in one pass, and there is no opportunity to verify intermediate steps.

In **agents**, CoT is embedded in the agent loop. The agent reasons about what to do, takes an action, observes the result, and reasons again. Each reasoning step is grounded in real data from tool use, not just the model's internal knowledge. This is profoundly more reliable.

```

# CoT in simple prompting: all reasoning in one pass
simple_cot = llm.generate(
    "Think step by step. What is the population growth rate of Tokyo?\n"
    "Step 1: I recall Tokyo's population was about 14 million in 2020.\n"
    "Step 2: ..."
    # All "steps" are generated from training data -- no verification
)

# CoT in agent: reasoning grounded in tool use
# Step 1: Agent thinks "I need current population data for Tokyo"
# Step 2: Agent searches the web for "Tokyo population 2025"
# Step 3: Agent observes: search returns 13.96 million
# Step 4: Agent thinks "I need historical data to calculate growth rate"
# Step 5: Agent searches for "Tokyo population 2015"
# Step 6: Agent observes: search returns 13.49 million
# Step 7: Agent thinks "Now I can calculate: (13.96 - 13.49) / 13.49 / 10 * 100"
# Step 8: Agent uses calculator: 0.35% annual growth rate
# Step 9: Agent provides verified, grounded answer

```

The agent version is slower and more expensive, but its answer is based on actual data rather than the model's potentially outdated or incorrect training knowledge. This is the fundamental trade-off in agent design: **accuracy and groundedness in exchange for latency and cost.**

The Plan-Execute-Evaluate Loop

The **Plan-Execute-Evaluate** loop adds an explicit evaluation step after execution. Instead of just planning and executing, the agent also assesses whether each step achieved its intended goal and whether the overall plan is still on track.

```
# Plan-Execute-Evaluate loop

class PlanExecuteEvaluateAgent:
    def __init__(self, llm, tools):
        self.llm = llm
        self.tools = tools
        self.plan = []
        self.results = []

    def run(self, goal: str) -> str:
        # PLAN
        self.plan = self.create_plan(goal)

        for i, step in enumerate(self.plan):
            # EXECUTE
            result = self.execute_step(step)
            self.results.append(result)

            # EVALUATE
            evaluation = self.evaluate_step(goal, step, result, i)

            if evaluation["status"] == "step_failed":
                # Retry with a modified approach
                retry_result = self.retry_step(step, result, evaluation["feedback"])
                self.results[-1] = retry_result

            elif evaluation["status"] == "plan_needs_revision":
                # Replan the remaining steps
                remaining_goal = evaluation["remaining_goal"]
                new_steps = self.create_plan(remaining_goal)
                self.plan = self.plan[i+1:] + new_steps

            elif evaluation["status"] == "goal_achieved_early":
                # No need to continue -- goal is already met
                break

        # Final evaluation
        return self.final_synthesis(goal)

    def evaluate_step(self, goal, step, result, step_index) -> dict:
        """Evaluate whether a step succeeded and whether the plan is on track."""
        evaluation = self.llm.generate(
            f"Overall goal: {goal}\n"
            f"Current step ({step_index + 1}/{len(self.plan)}): {step}\n"
            f"Step result: {result}\n"
            f"All results so far: {self.results}\n\n"
            f"Evaluate:\n"
```

```

        f"1. Did this step succeed in its objective?\n"
        f"2. Is the overall plan still appropriate, or does it need revision?\n"
        f"3. Has the overall goal already been achieved?\n"
        f"Respond with status: step_succeeded, step_failed, "
        f"plan_needs_revision, or goal_achieved_early."
    )
    return parse_evaluation(evaluation)

```

The evaluation step is the difference between an agent that plows ahead blindly and one that adapts intelligently. It catches situations where a step technically succeeded but produced unhelpful results, where new information changes the optimal plan, or where the goal has been achieved before all planned steps are complete.

Handling Failures: Replanning When Things Go Wrong

Failure handling is where agents either prove their value or fall apart. In the real world, tools fail, search results are unhelpful, APIs return errors, and intermediate results are not what you expected. A robust agent must detect failures, diagnose them, and adapt.

```

# Comprehensive failure handling

class ResilientAgent:
    def __init__(self, llm, tools):
        self.llm = llm
        self.tools = tools
        self.failure_log = []

    def execute_with_recovery(self, step: str, max_retries: int = 3) -> dict:
        """Execute a step with automatic failure recovery."""

        for attempt in range(max_retries):
            result = self.execute_step(step)

            if result["success"]:
                return result

        # Log the failure
        self.failure_log.append({
            "step": step,
            "attempt": attempt + 1,
            "error": result.get("error", "Unknown error")
        })

        if attempt < max_retries - 1:
            # Ask the LLM to diagnose and suggest an alternative approach
            recovery_plan = self.llm.generate(
                f"Step attempted: {step}\n"
                f"Error encountered: {result['error']}\n"
                f"Previous attempts: {self.failure_log}\n\n"
                f"Diagnose the problem and suggest an alternative approach. "
                f"Do NOT suggest retrying the exact same thing."
            )

```

```

    )

    # Use the alternative approach
    step = recovery_plan

# All retries exhausted
return {
    "success": False,
    "error": f"Failed after {max_retries} attempts",
    "failure_log": self.failure_log,
    "fallback": self.generate_fallback(step)
}

def generate_fallback(self, failed_step: str) -> str:
    """Generate a graceful fallback when a step cannot be completed."""
    return self.llm.generate(
        f"The following step could not be completed: {failed_step}\n"
        f"Failures: {self.failure_log}\n\n"
        f"Provide the best possible answer using only the information "
        f"you already have. Clearly note what information is missing."
    )

```

Common failure patterns and their solutions:

- **Tool returns empty results:** Try rephrasing the query. If searching, use different keywords. If querying a database, broaden the filter criteria.
- **Tool returns an error:** Parse the error message. Common issues include authentication failures (check credentials), rate limits (wait and retry), and malformed inputs (validate parameters).
- **Results are unhelpful:** The tool worked but the output does not advance the task. Try a different tool or a different approach entirely.
- **The plan itself is wrong:** Sometimes the agent's initial decomposition is flawed. Implement periodic "step back" evaluations where the agent reassesses its overall approach.

Task Graphs and Dependencies

For complex tasks, a linear sequence of steps is insufficient. Some steps can run in parallel, others have strict dependencies, and some are conditional. **Task graphs** model these relationships explicitly.

```

# Task graph implementation

from enum import Enum
from dataclasses import dataclass, field

class TaskStatus(Enum):
    PENDING = "pending"
    READY = "ready" # All dependencies met
    IN_PROGRESS = "in_progress"
    COMPLETED = "completed"
    FAILED = "failed"
    SKIPPED = "skipped"

@dataclass

```

```

class TaskNode:
    id: str
    description: str
    dependencies: list = field(default_factory=list) # List of task IDs
    status: TaskStatus = TaskStatus.PENDING
    result: str = None

class TaskGraph:
    def __init__(self):
        self.tasks = {}

    def add_task(self, task: TaskNode):
        self.tasks[task.id] = task

    def get_ready_tasks(self) -> list:
        """Return all tasks whose dependencies are met."""
        ready = []
        for task in self.tasks.values():
            if task.status != TaskStatus.PENDING:
                continue
            deps_met = all(
                self.tasks[dep].status == TaskStatus.COMPLETED
                for dep in task.dependencies
            )
            if deps_met:
                task.status = TaskStatus.READY
                ready.append(task)
        return ready

    def is_complete(self) -> bool:
        return all(
            t.status in (TaskStatus.COMPLETED, TaskStatus.SKIPPED)
            for t in self.tasks.values()
        )

# Build a task graph for a research report
graph = TaskGraph()
graph.add_task(TaskNode("search_eu", "Search for EU AI regulations", []))
graph.add_task(TaskNode("search_us", "Search for US AI regulations", []))
graph.add_task(TaskNode("search_compare", "Search for expert comparisons", []))
graph.add_task(TaskNode("analyze", "Compare EU vs US findings",
                        ["search_eu", "search_us", "search_compare"]))
graph.add_task(TaskNode("write_report", "Write the briefing document", ["analyze"]))

# Execution: search_eu, search_us, search_compare can run in parallel
# analyze waits for all three searches
# write_report waits for analyze

```

Task graphs enable **parallel execution** (running independent steps simultaneously), **conditional branching** (skipping steps based on results), and **dynamic graph modification** (adding new tasks based on intermediate findings).

Full Example: Research, Synthesize, Write Report Agent

Here is a complete implementation of an agent that researches a topic, synthesizes findings, and produces a written report.

```
# Complete research-to-report agent

class ResearchReportAgent:
    def __init__(self, llm, tools):
        self.llm = llm
        self.tools = tools
        self.working_memory = WorkingMemory()

    def run(self, research_question: str) -> str:
        """Execute the full research-to-report pipeline."""

        # Phase 1: Plan the research
        research_plan = self.plan_research(research_question)

        # Phase 2: Execute research queries
        findings = self.execute_research(research_plan)

        # Phase 3: Synthesize findings
        synthesis = self.synthesize_findings(research_question, findings)

        # Phase 4: Write the report
        report = self.write_report(research_question, synthesis)

        # Phase 5: Review and revise
        final_report = self.review_and_revise(research_question, report)

        return final_report

    def plan_research(self, question: str) -> list:
        """Determine what searches and data gathering are needed."""
        plan = self.llm.generate(
            f"I need to research: {question}\n\n"
            f"What specific searches, data lookups, or information gathering "
            f"steps are needed? List 5-8 specific queries to run.\n"
            f"For each, specify what tool to use and what information to look for."
        )
        return parse_research_plan(plan)

    def execute_research(self, plan: list) -> list:
        """Execute each research step and collect findings."""
        findings = []

        for step in plan:
            # Use the appropriate tool
            if step["tool"] == "web_search":
                result = self.tools.execute("web_search", {"query": step["query"]})
```

```

        elif step["tool"] == "search_arxiv":
            result = self.tools.execute("search_arxiv", {"query":
step["query"]})
        else:
            result = self.tools.execute(step["tool"], step.get("params", {}))

        # Have the LLM extract key findings from the raw result
        extracted = self.llm.generate(
            f"Research question: {step['query']}\n"
            f"Raw results:\n{result}\n\n"
            f"Extract the key facts, statistics, and insights. "
            f"Note the source for each claim."
        )

        findings.append({
            "query": step["query"],
            "raw_result": result,
            "extracted": extracted
        })

        self.working_memory.add_finding(extracted, source=step["query"])

    return findings

def synthesize_findings(self, question: str, findings: list) -> str:
    """Synthesize all findings into a coherent analysis."""
    findings_text = "\n\n".join([
        f"Finding {i+1} (from: {f['query']}):\n{f['extracted']}"
        for i, f in enumerate(findings)
    ])

    synthesis = self.llm.generate(
        f"Research question: {question}\n\n"
        f"All findings:\n{findings_text}\n\n"
        f"Synthesize these findings into a coherent analysis:\n"
        f"1. Identify the main themes\n"
        f"2. Note any contradictions between sources\n"
        f"3. Highlight the strongest evidence\n"
        f"4. Identify gaps in the research"
    )

    return synthesis

def write_report(self, question: str, synthesis: str) -> str:
    """Write a structured report from the synthesis."""
    report = self.llm.generate(
        f"Write a professional briefing report on: {question}\n\n"
        f"Based on this analysis:\n{synthesis}\n\n"
        f"Structure:\n"
        f"- Executive Summary (2-3 paragraphs)\n"
        f"- Key Findings (bulleted)\n"
        f"- Detailed Analysis (3-5 sections)\n"
        f"- Recommendations\n"
    )

```

```

        f"- Sources and Limitations"
    )
    return report

def review_and_revise(self, question: str, report: str) -> str:
    """Self-review the report and make improvements."""
    critique = self.llm.generate(
        f"Review this report critically. The original question was: {question}\n\n"
        f"Report:\n{report}\n\n"
        f"Identify:\n"
        f"1. Factual claims that need stronger evidence\n"
        f"2. Logical gaps or unsupported conclusions\n"
        f"3. Missing perspectives\n"
        f"4. Clarity and structure issues"
    )

    revised = self.llm.generate(
        f"Revise this report based on the following critique:\n\n"
        f"Critique:\n{critique}\n\n"
        f"Original report:\n{report}\n\n"
        f"Produce an improved version that addresses each critique point."
    )
    return revised

```

Key Takeaways

- **Task decomposition** is the foundation of agent planning. Complex goals must be broken into concrete, executable steps.
- **Three planning strategies** serve different needs: top-down (structured but rigid), iterative (adaptive but can wander), hierarchical (best of both).
- **Agent CoT is grounded** in real tool results, making it fundamentally more reliable than standalone CoT prompting.
- The **Plan-Execute-Evaluate** loop adds critical quality control by assessing results after each step.
- **Failure handling** distinguishes production agents from prototypes. Build in retry logic, alternative approaches, and graceful degradation.
- **Task graphs** model complex dependencies and enable parallel execution of independent steps.
- A **complete research pipeline** combines planning, iterative research, synthesis, writing, and self-review into a cohesive agent workflow.

Chapter 6: Multi-Agent Systems

When One Agent Isn't Enough

Single agents work well for many tasks, but they hit limits when problems become sufficiently complex, require genuinely different expertise, or benefit from adversarial review. Here are the situations where multi-agent systems earn their added complexity.

Tool set overload. When an agent needs 30+ tools, its ability to select the right tool degrades. Splitting tools across specialized agents, each with 5-10 focused tools, restores selection accuracy. A research agent with web

search tools and a coding agent with code execution tools will each perform better than a single agent juggling everything.

Conflicting personas. Asking one agent to be both creative and critical is asking it to argue with itself. A writer agent and a reviewer agent, operating as separate entities, produce better results than one agent trying to wear both hats. The reviewer genuinely critiques the writer's output because it has a different objective.

Task parallelism. Some tasks have independent subtasks that can execute simultaneously. Multiple agents can work in parallel, each handling a different subtask, reducing total completion time.

Quality through debate. Multiple agents with different perspectives can debate an issue, catching errors and biases that a single agent would miss. This is particularly valuable for high-stakes decisions.

Scalability. As tasks grow in scope, a single agent's context window becomes a bottleneck. Multiple agents, each managing their own context, can collectively handle much larger problems.

```
# When to use multi-agent: a decision framework

def should_use_multi_agent(task_description: str) -> dict:
    criteria = {
        "tool_count": count_required_tools(task_description),
        "distinct_phases": identify_phases(task_description),
        "needs_review": requires_quality_review(task_description),
        "parallelizable": has_independent_subtasks(task_description),
        "context_size": estimate_context_requirements(task_description),
    }

    recommendation = "single_agent" # Default
    reasons = []

    if criteria["tool_count"] > 20:
        recommendation = "multi_agent"
        reasons.append("Too many tools for effective single-agent selection")

    if criteria["distinct_phases"] >= 3:
        recommendation = "multi_agent"
        reasons.append(f"{criteria['distinct_phases']} distinct phases benefit from specialization")

    if criteria["needs_review"]:
        recommendation = "multi_agent"
        reasons.append("Quality-critical task benefits from separate reviewer agent")

    return {"recommendation": recommendation, "reasons": reasons}
```

Agent Roles: Researcher, Writer, Critic, Coordinator

In multi-agent systems, each agent takes on a specific **role** that defines its expertise, tools, and behavior. Common roles include:

Researcher. Gathers information from external sources. Has access to search tools, databases, and knowledge bases. Optimized for finding relevant, accurate information.

Analyst. Processes and interprets data. Has access to code execution, calculators, and data processing tools. Focuses on extracting insights from raw information.

Writer. Produces polished output. Has access to file writing and formatting tools. Specializes in clear, well-structured communication.

Critic / Reviewer. Evaluates the work of other agents. Has no tools other than its reasoning ability. Focuses on finding errors, gaps, and improvements.

Coordinator / Orchestrator. Manages the overall workflow. Decides which agent to invoke, passes information between agents, and determines when the task is complete. May or may not be an LLM-based agent itself.

```
# Defining agent roles

class AgentRole:
    def __init__(self, name: str, system_prompt: str, tools: list, model: str =
None):
        self.name = name
        self.system_prompt = system_prompt
        self.tools = tools
        self.model = model # Can use different models for different roles

# Define specialized roles
researcher = AgentRole(
    name="Researcher",
    system_prompt=(
        "You are a thorough research specialist. Your job is to find accurate, "
        "relevant information. Always cite your sources. If information is
uncertain, "
        "say so. Search multiple sources to verify important claims. "
        "You do NOT write reports -- you gather raw findings."
    ),
    tools=[web_search, arxiv_search, wikipedia_search],
    model="claude-sonnet-4-20250514" # Fast model for many searches
)

writer = AgentRole(
    name="Writer",
    system_prompt=(
        "You are an expert technical writer. Given research findings, "
        "you produce clear, well-structured documents. Focus on: "
        "logical flow, clear explanations, appropriate detail level, "
        "and professional tone. You do NOT conduct research -- you write."
    ),
    tools=[write_file, read_file],
    model="claude-opus-4-20250514" # Best model for quality writing
)

critic = AgentRole(
```

```

name="Critic",
system_prompt=(
    "You are a rigorous reviewer. Examine the given work for: "
    "factual accuracy, logical consistency, missing perspectives, "
    "unclear explanations, and unsupported claims. Be specific "
    "in your criticisms and suggest concrete improvements. "
    "Rate overall quality on a scale of 1-10."
),
tools=[], # Critic only reasons, no tools
model="claude-opus-4-20250514"
)

coordinator = AgentRole(
    name="Coordinator",
    system_prompt=(
        "You are the project coordinator. Your job is to: "
        "1. Break the task into phases for specialized agents. "
        "2. Pass the right information to each agent. "
        "3. Review intermediate results and decide next steps. "
        "4. Determine when the task is complete."
    ),
    tools=[],
    model="claude-sonnet-4-20250514"
)

```

Communication Patterns: Sequential, Parallel, Hierarchical, Debate

How agents communicate and pass information defines the system's behavior. Four primary patterns cover most use cases.

Sequential (Pipeline). Agent A finishes its work, passes results to Agent B, who passes to Agent C, and so on. Like an assembly line. Simple to implement, easy to debug, but slow because there is no parallelism.

```
[Researcher] -> [Analyst] -> [Writer] -> [Reviewer]
```

Parallel (Fan-Out/Fan-In). Multiple agents work simultaneously on different subtasks. A coordinator fans out work, collects results, and synthesizes. Fast but requires careful coordination.

```

-> [Researcher A] ->
[Coordinator] -> [Researcher B] -> [Coordinator] -> [Writer]
-> [Researcher C] ->

```

Hierarchical (Manager/Worker). A manager agent delegates to worker agents, reviews their output, and may reassign or request revisions. Scales well but depends heavily on the manager's quality.

```

[Manager]
 /   |   \
[Worker A] [Worker B] [Worker C]

```

Debate. Two or more agents argue opposing positions. A judge agent evaluates the arguments and determines the best answer. Excellent for reducing errors and biases but expensive (many LLM calls).

```
[Advocate A] <-> [Advocate B]
      \           /
      [Judge]
```

```
# Implementing the four communication patterns

# Pattern 1: Sequential Pipeline
class SequentialPipeline:
    def __init__(self, agents: list):
        self.agents = agents # Ordered list of agent roles

    def run(self, initial_input: str) -> str:
        current_output = initial_input
        for agent in self.agents:
            current_output = agent.run(current_output)
        return current_output

# Pattern 2: Parallel Fan-Out/Fan-In
class ParallelFanOut:
    def __init__(self, coordinator, workers: list):
        self.coordinator = coordinator
        self.workers = workers

    def run(self, task: str) -> str:
        # Fan-out: coordinator creates subtasks
        subtasks = self.coordinator.create_subtasks(task, len(self.workers))

        # Execute in parallel (using async or threading)
        import concurrent.futures
        with concurrent.futures.ThreadPoolExecutor() as executor:
            futures = {
                executor.submit(worker.run, subtask): worker
                for worker, subtask in zip(self.workers, subtasks)
            }
            results = {}
            for future in concurrent.futures.as_completed(futures):
                worker = futures[future]
                results[worker.name] = future.result()

        # Fan-in: coordinator synthesizes results
        return self.coordinator.synthesize(task, results)

# Pattern 3: Hierarchical Manager/Worker
class HierarchicalSystem:
    def __init__(self, manager, workers: dict):
        self.manager = manager
        self.workers = workers # {name: agent}

    def run(self, task: str) -> str:
        plan = self.manager.plan(task)
```

```

for assignment in plan:
    worker_name = assignment["worker"]
    worker_task = assignment["task"]

    result = self.workers[worker_name].run(worker_task)

    # Manager reviews the result
    review = self.manager.review(worker_task, result)

    if review["needs_revision"]:
        result = self.workers[worker_name].run(
            f"Revise based on feedback: {review['feedback']}\n"
            f"Original output: {result}"
        )

    plan.update_with_result(assignment["id"], result)

return self.manager.compile_final(plan)

# Pattern 4: Debate
class DebateSystem:
    def __init__(self, advocate_a, advocate_b, judge):
        self.advocate_a = advocate_a
        self.advocate_b = advocate_b
        self.judge = judge

    def run(self, question: str, rounds: int = 3) -> str:
        debate_history = []

        for round_num in range(rounds):
            # Advocate A presents argument
            a_context = f"Question: {question}\nDebate history: {debate_history}"
            a_argument = self.advocate_a.run(
                f"{a_context}\nPresent your argument. Address any counterpoints
raised."
            )
            debate_history.append({"speaker": "A", "argument": a_argument})

            # Advocate B responds
            b_context = f"Question: {question}\nDebate history: {debate_history}"
            b_argument = self.advocate_b.run(
                f"{b_context}\nPresent your counterargument. Challenge weak points."
            )
            debate_history.append({"speaker": "B", "argument": b_argument})

        # Judge evaluates
        verdict = self.judge.run(
            f"Question: {question}\n"
            f"Full debate:\n{format_debate(debate_history)}\n\n"
            f"Evaluate both sides. Which arguments are stronger and why? "
            f"Provide your reasoned verdict."

```



```
)  
return verdict
```

Orchestrating Multiple Agents

Orchestration is the mechanism that coordinates multiple agents. The orchestrator decides which agent runs next, what information to pass, when to retry, and when the overall task is complete.

```
# A flexible agent orchestrator  
  
class AgentOrchestrator:  
    def __init__(self, agents: dict, llm=None):  
        self.agents = agents # {name: Agent}  
        self.llm = llm # Optional LLM for intelligent routing  
        self.execution_log = []  
  
    def run(self, task: str, strategy: str = "auto") -> str:  
        if strategy == "sequential":  
            return self.run_sequential(task)  
        elif strategy == "parallel":  
            return self.run_parallel(task)  
        elif strategy == "auto":  
            return self.run_intelligent(task)  
  
    def run_intelligent(self, task: str) -> str:  
        """LLM-driven orchestration -- the orchestrator reasons about what to do."""  
        context = {  
            "task": task,  
            "available_agents": {  
                name: agent.description for name, agent in self.agents.items()  
            },  
            "completed_steps": [],  
            "results": {}  
        }  
  
        for step in range(20):  
            # Ask the orchestrator LLM what to do next  
            decision = self.llm.generate(  
                f"Task: {context['task']}\n"  
                f"Available agents: {json.dumps(context['available_agents'])}\n"  
                f"Completed steps: {context['completed_steps']}\n"  
                f"Results so far: {json.dumps(context['results'])}\n\n"  
                f"What should happen next? Options:\n"  
                f"1. DELEGATE: <agent_name> | <subtask description>\n"  
                f"2. COMPLETE: <final synthesis>\n"  
                f"Choose one and explain why."  
            )  
  
            if "COMPLETE" in decision:  
                return extract_completion(decision)
```

```

# Parse delegation instruction
agent_name, subtask = parse_delegation(decision)

if agent_name in self.agents:
    # Include relevant context from previous results
    agent_input = self.build_agent_input(subtask, context["results"])
    result = self.agents[agent_name].run(agent_input)

    context["completed_steps"].append({
        "agent": agent_name,
        "subtask": subtask,
        "step": step
    })
    context["results"][f"step_{step}_{agent_name}"] = result

    self.execution_log.append({
        "step": step,
        "agent": agent_name,
        "subtask": subtask,
        "result_preview": result[:200]
    })

return "Orchestration reached maximum steps."

def build_agent_input(self, subtask: str, previous_results: dict) -> str:
    """Build the input for an agent, including relevant context."""
    relevant = self.llm.generate(
        f"Subtask: {subtask}\n"
        f"Available results: {list(previous_results.keys())}\n"
        f"Which previous results are relevant to this subtask? List them."
    )

    context_parts = [f"Your task: {subtask}"]
    for key in previous_results:
        if key in relevant:
            context_parts.append(f"\nContext from {key}: {previous_results[key]}")

    return "\n".join(context_parts)

```

Debate and Self-Critique Patterns

The **debate pattern** is one of the most effective ways to improve agent output quality. By having agents argue different positions, you surface weaknesses, errors, and blind spots that a single agent would miss.

```

# Self-critique: one agent reviews its own work

class SelfCritiqueAgent:
    def __init__(self, llm, tools):
        self.llm = llm
        self.tools = tools

```

```

def run_with_critique(self, task: str, critique_rounds: int = 2) -> str:
    # Initial attempt
    draft = self.run(task)

    for round_num in range(critique_rounds):
        # Critique the current draft
        critique = self.llm.generate(
            f"You are a critical reviewer. Examine this output:\n\n"
            f"Task: {task}\n"
            f"Output:\n{draft}\n\n"
            f"Identify:\n"
            f"1. Factual errors or unsupported claims\n"
            f"2. Logical inconsistencies\n"
            f"3. Missing important information\n"
            f"4. Unclear or confusing sections\n"
            f"5. Overall quality rating (1-10)\n"
            f"Be specific and constructive."
        )

        # Check if quality is sufficient
        quality_rating = extract_rating(critique)
        if quality_rating >= 8:
            break # Good enough

        # Revise based on critique
        draft = self.llm.generate(
            f"Original task: {task}\n"
            f"Current draft:\n{draft}\n"
            f"Critique:\n{critique}\n\n"
            f"Revise the output to address each critique point. "
            f"Improve quality while preserving what already works well."
        )

    return draft

```

Multi-agent debate: separate agents argue positions

```

class StructuredDebate:
    def __init__(self, pro_agent, con_agent, judge_agent):
        self.pro = pro_agent
        self.con = con_agent
        self.judge = judge_agent

    def debate(self, proposition: str, rounds: int = 3) -> dict:
        history = []

        for r in range(rounds):
            # Pro agent argues in favor
            pro_arg = self.pro.run(
                f"Proposition: {proposition}\n"
                f"Previous arguments: {history}\n"

```

```

        f"Round {r+1}: Argue IN FAVOR. Address counterarguments."
    )
    history.append({"side": "pro", "round": r+1, "argument": pro_arg})

    # Con agent argues against
    con_arg = self.con.run(
        f"Proposition: {proposition}\n"
        f"Previous arguments: {history}\n"
        f"Round {r+1}: Argue AGAINST. Challenge the pro arguments."
    )
    history.append({"side": "con", "round": r+1, "argument": con_arg})

    # Judge evaluates
    verdict = self.judge.run(
        f"Proposition: {proposition}\n"
        f"Complete debate transcript:\n{format_debate(history)}\n\n"
        f"Evaluate each argument's strength. Identify the most compelling "
        f"points from each side. Render your verdict with detailed reasoning."
    )

    return {
        "proposition": proposition,
        "debate_history": history,
        "verdict": verdict
    }

```

Frameworks: CrewAI, AutoGen, LangGraph (comparison)

Feature	CrewAI	AutoGen	LangGraph
Core concept	Agents with roles, tasks, crews	Conversational agents	Graph-based state machines
Strength	Multi-agent collaboration	Agent-to-agent dialogue	Complex, stateful workflows
Learning curve	Low	Medium	High
Flexibility	Medium	High	Very high
Best for	Role-based teams	Research/debate	Production workflows
State management	Basic	Conversation-based	Graph state, checkpointing
Human-in-loop	Supported	Core feature	Supported

CrewAI excels at defining agent teams where each member has a clear role. You define agents, tasks, and how they collaborate. The API is intuitive and maps well to how people think about team work.

AutoGen pioneered conversational multi-agent systems. Agents literally talk to each other, with each message being a full LLM response. This is natural for debate, brainstorming, and iterative refinement tasks.

LangGraph provides the most control through a graph-based state machine. You define nodes (agent steps) and edges (transitions), with full control over state, branching, and loops. It is the most powerful but also the most complex.

```
# CrewAI example: Research team
from crewai import Agent, Task, Crew

researcher = Agent(
    role="Senior Research Analyst",
    goal="Find comprehensive, accurate information on the given topic",
    backstory="You are a veteran researcher with 20 years of experience...",
    tools=[web_search_tool, arxiv_tool]
)

writer = Agent(
    role="Technical Writer",
    goal="Produce clear, well-structured reports from research findings",
    backstory="You are an award-winning technical writer...",
    tools=[file_tool]
)

research_task = Task(
    description="Research the current state of AI agents in production use",
    agent=researcher,
    expected_output="A detailed research brief with key findings and sources"
)

writing_task = Task(
    description="Write a comprehensive report based on the research findings",
    agent=writer,
    expected_output="A polished report in markdown format",
    context=[research_task] # Writing depends on research
)

crew = Crew(
    agents=[researcher, writer],
    tasks=[research_task, writing_task],
    verbose=True
)

result = crew.kickoff()
```

Building a Multi-Agent Research Team Step by Step

Let us build a complete multi-agent research system from scratch, without relying on a framework, to understand how the pieces fit together.

```
# Complete multi-agent research team

class ResearchTeam:
```

```

def __init__(self, llm_provider):
    self.llm = llm_provider

    # Define specialized agents
    self.researcher = self.create_agent(
        "Researcher",
        "You find information. Search thoroughly. Verify claims with multiple
sources. "
        "Report raw findings with sources. Do not interpret or analyze.",
        [web_search, arxiv_search]
    )

    self.analyst = self.create_agent(
        "Analyst",
        "You analyze information. Find patterns, contradictions, and insights. "
        "Use data to support conclusions. Be quantitative where possible.",
        [python_execute, calculator]
    )

    self.writer = self.create_agent(
        "Writer",
        "You write polished reports. Structure content logically. "
        "Use clear language. Cite sources. Make complex topics accessible.",
        [write_file, read_file]
    )

    selfReviewer = self.create_agent(
        "Reviewer",
        "You review documents for quality. Check facts, logic, clarity, "
        "completeness. Score quality 1-10. Provide specific improvement
suggestions.",
        []
    )

def create_agent(self, name, instructions, tools):
    return {
        "name": name,
        "instructions": instructions,
        "tools": tools,
        "run": lambda task, inst=instructions, t=tools: self.run_agent(inst, t,
task)
    }

def run_agent(self, instructions: str, tools: list, task: str) -> str:
    """Run a single agent to completion on a task."""
    messages = [{"role": "user", "content": task}]

    for step in range(10):
        response = self.llm.generate(
            system=instructions,
            messages=messages,
            tools=[t.schema for t in tools] if tools else None

```

```

    )

    if response.has_tool_call():
        result = execute_tool(response.tool_call, tools)
        messages.append({"role": "assistant", "content": response.content})
        messages.append(tool_result_message(response.tool_call.id, result))
    else:
        return response.text

    return "Agent reached step limit."

def execute_project(self, research_question: str) -> str:
    """Run the full research team pipeline."""

    print(f"[Coordinator] Starting research on: {research_question}")

    # Step 1: Researcher gathers information
    print("[Coordinator] Delegating to Researcher...")
    research_findings = self.researcher["run"](
        f"Research the following question thoroughly:\n{research_question}\n\n"
        f"Search for at least 5 different sources. Include recent data and"
expert opinions."
    )
    print(f"[Researcher] Complete. Found {len(research_findings)} characters of"
findings.")

    # Step 2: Analyst processes findings
    print("[Coordinator] Delegating to Analyst...")
    analysis = self.analyst["run"](
        f"Analyze these research findings:\n\n{research_findings}\n\n"
        f"Identify key trends, quantify where possible, note contradictions, "
        f"and draw evidence-based conclusions."
    )
    print(f"[Analyst] Complete. Analysis ready.")

    # Step 3: Writer produces report
    print("[Coordinator] Delegating to Writer...")
    draft_report = self.writer["run"](
        f"Write a professional report on: {research_question}\n\n"
        f"Research findings:\n{research_findings}\n\n"
        f"Analysis:\n{analysis}\n\n"
        f"Produce a complete report with executive summary, key findings, "
        f"detailed analysis, and recommendations."
    )
    print(f"[Writer] Complete. Draft report ready.")

    # Step 4: Reviewer checks quality
    print("[Coordinator] Delegating to Reviewer...")
    review = self.reviewer["run"](
        f"Review this report critically:\n\n{draft_report}\n\n"
        f"Original question: {research_question}\n"
        f"Source material: {research_findings[:2000]}...\n\n"

```

```

        f"Evaluate quality, accuracy, completeness. Score 1-10."
    )
    print(f"[Reviewer] Complete. Review ready.")

    # Step 5: Writer revises based on review
    quality_score = extract_rating(review)
    if quality_score < 8:
        print(f"[Coordinator] Quality score {quality_score}/10. Requesting
revision...")
        final_report = self.writer["run"](
            f"Revise this report based on reviewer feedback:\n\n"
            f"Current draft:\n{draft_report}\n\n"
            f"Reviewer feedback:\n{review}\n\n"
            f"Address each critique point while preserving what works."
        )
    else:
        print(f"[Coordinator] Quality score {quality_score}/10. Report
approved.")
        final_report = draft_report

    print("[Coordinator] Research project complete.")
    return final_report

# Usage
team = ResearchTeam(llm_provider=anthropic_client)
report = team.execute_project(
    "What are the most effective strategies for deploying AI agents in enterprise
environments?"
)

```

Key Takeaways

- Use multi-agent systems when tasks require **specialized expertise, quality review through debate, parallel execution**, or when the **tool set is too large** for a single agent.
- Define clear **agent roles** with specific system prompts, tool sets, and objectives. Specialization improves quality.
- Four **communication patterns** cover most needs: sequential pipelines, parallel fan-out/fan-in, hierarchical manager/worker, and debate.
- **Orchestration** can be rule-based (predetermined flow) or intelligent (LLM-driven routing). Start with rule-based and add intelligence when needed.
- **Debate and self-critique** are among the most effective patterns for improving output quality.
- **Frameworks** (CrewAI, AutoGen, LangGraph) provide structure but are not required. Understanding the underlying patterns is more important than learning a specific framework.
- **Start simple**. Build the multi-agent system with explicit, hardcoded orchestration first. Add LLM-driven orchestration only after the basic flow works correctly.

Chapter 7: Building Production-Ready Agents

From Prototype to Production: What Changes

The gap between a prototype agent that works in a demo and a production agent that handles real users is enormous. Here is what changes.

Reliability. A prototype can fail 20% of the time and you just re-run it. A production agent must handle failures gracefully, recover automatically, and never leave users stuck. This means comprehensive error handling, retry logic, fallback strategies, and timeouts at every level.

Scale. A prototype handles one request at a time from a developer's laptop. A production agent handles hundreds of concurrent requests from real users. This means async execution, request queuing, connection pooling, and horizontal scaling.

Observability. When a prototype breaks, you add print statements and debug. When a production agent breaks at 3 AM, you need logs, traces, metrics, and alerts that tell you what happened and why. You cannot add print statements to an agent serving live users.

Cost. A prototype that burns \$5 per run is fine for testing. A production agent that burns \$5 per run at 10,000 requests per day costs \$50,000 per day. Cost optimization becomes critical and that means caching, model selection, prompt optimization, and early termination strategies.

Safety. A prototype running on your machine with your data has limited blast radius. A production agent with access to production databases, customer data, and external APIs can cause real harm. Safety guardrails, permission systems, and human oversight become essential.

```
# Prototype vs. production: the code difference

# PROTOTYPE: simple, no error handling, no logging
def prototype_agent(question):
    response = llm.generate(question, tools=tools)
    if response.has_tool_call():
        result = execute_tool(response.tool_call)
        return llm.generate(question + str(result))
    return response.text

# PRODUCTION: comprehensive handling
class ProductionAgent:
    def __init__(self, config: AgentConfig):
        self.llm = LLMClient(
            model=config.model,
            timeout=config.llm_timeout,
            retry_config=config.retry_config,
            rate_limiter=RateLimiter(config.max_requests_per_minute)
        )
        self.tools = ToolSet(config.tools, timeout=config.tool_timeout)
        self.logger = StructuredLogger(config.log_config)
        self.tracer = Tracer(config.tracing_endpoint)
        self.cost_tracker = CostTracker(config.cost_limits)
        self.guardrails = GuardrailSet(config.safety_rules)
        self.metrics = MetricsCollector()

    async def run(self, request: AgentRequest) -> AgentResponse:
        trace_id = self.tracer.start_trace(request)
        start_time = time.time()
```

```

try:
    # Check guardrails before starting
    self.guardrails.validate_input(request)

    # Check cost budget
    self.cost_tracker.check_budget(request.user_id)

    result = await self._execute_agent_loop(request, trace_id)

    # Check guardrails on output
    self.guardrails.validate_output(result)

    # Record metrics
    self.metrics.record(
        duration=time.time() - start_time,
        steps=result.step_count,
        tokens_used=result.total_tokens,
        cost=result.total_cost,
        success=True
    )

    return AgentResponse(
        result=result.final_output,
        trace_id=trace_id,
        metadata=result.metadata
    )

except GuardrailViolation as e:
    self.logger.warn("Guardrail violation", trace_id=trace_id,
violation=str(e))
    return AgentResponse(error="Request could not be processed safely.")

except CostLimitExceeded as e:
    self.logger.warn("Cost limit exceeded", trace_id=trace_id)
    return AgentResponse(error="Request exceeded cost limits.")

except Exception as e:
    self.logger.error("Agent execution failed", trace_id=trace_id,
error=str(e))
    self.metrics.record(success=False, error_type=type(e).__name__)
    return AgentResponse(error="An unexpected error occurred. Please try
again.")

```

Observability: Logging, Tracing, Monitoring

You cannot fix what you cannot see. Agent observability means capturing enough information to understand what the agent did, why it did it, how long it took, and what went wrong when things break.

Structured Logging captures every significant event in a parseable format. Not print statements, but structured JSON logs with consistent fields.

```

# Structured logging for agents

import json
import time

class AgentLogger:
    def __init__(self, agent_name: str, log_destination: str = "stdout"):
        self.agent_name = agent_name
        self.destination = log_destination

    def log(self, event: str, level: str = "info", **kwargs):
        log_entry = {
            "timestamp": time.time(),
            "agent": self.agent_name,
            "event": event,
            "level": level,
            **kwargs
        }

        if self.destination == "stdout":
            print(json.dumps(log_entry))
        else:
            # Send to log aggregation service (Datadog, CloudWatch, etc.)
            send_to_log_service(log_entry)

    def log_step(self, step_num: int, thought: str, action: str,
                  result: str, tokens_used: int, duration_ms: float):
        self.log(
            "agent_step",
            step=step_num,
            thought_preview=thought[:200],
            action=action,
            result_preview=result[:200],
            tokens=tokens_used,
            duration_ms=duration_ms
        )

    def log_tool_call(self, tool_name: str, params: dict,
                      result: str, success: bool, duration_ms: float):
        self.log(
            "tool_call",
            tool=tool_name,
            params=params,
            result_preview=result[:200],
            success=success,
            duration_ms=duration_ms
        )

    def log_completion(self, total_steps: int, total_tokens: int,
                       total_cost: float, total_duration_ms: float, success: bool):
        self.log(

```

```

        "agent_complete",
        level="info" if success else "error",
        total_steps=total_steps,
        total_tokens=total_tokens,
        total_cost_usd=total_cost,
        total_duration_ms=total_duration_ms,
        success=success
    )

```

Distributed Tracing connects all the steps of an agent execution into a single trace that you can follow from start to finish. Tools like LangSmith, Arize Phoenix, and custom OpenTelemetry setups provide this capability.

Metrics and Dashboards track aggregate performance: average response time, step count distribution, tool failure rates, cost per request, and success rates. Set up alerts for anomalies -- if the average cost per request doubles overnight, you want to know immediately.

```

# Key metrics to track

class AgentMetrics:
    def __init__(self):
        self.metrics = {
            # Latency
            "response_time_p50": Histogram(),
            "response_time_p99": Histogram(),

            # Cost
            "cost_per_request": Histogram(),
            "tokens_per_request": Histogram(),

            # Quality
            "success_rate": Counter(),
            "failure_rate": Counter(),
            "steps_per_request": Histogram(),

            # Tools
            "tool_call_count": Counter(labels=["tool_name"]),
            "tool_failure_rate": Counter(labels=["tool_name"]),
            "tool_latency": Histogram(labels=["tool_name"]),

            # Safety
            "guardrail_triggers": Counter(labels=["rule_name"]),
            "human_escalations": Counter(),
        }

    def create_dashboard(self):
        """Define dashboard panels for monitoring."""
        return {
            "panels": [
                {
                    "title": "Request Volume",
                    "metric": "request_count",
                    "type": "timeseries",
                },
                {
                    "title": "Success Rate",
                    "metric": "success_rate",
                    "type": "gauge",
                    "threshold": 0.95,
                },
            ],
        }

```

```

        {"title": "P99 Latency", "metric": "response_time_p99", "type":
"timeseries"},
        {"title": "Cost per Request", "metric": "cost_per_request", "type":
"timeseries"},
        {"title": "Tool Failure Rates", "metric": "tool_failure_rate",
"type": "bar_chart"},
        {"title": "Steps per Request", "metric": "steps_per_request",
"type": "histogram"},
        {"title": "Guardrail Triggers", "metric": "guardrail_triggers",
"type": "table"},
    ]
}

```

Safety and Guardrails: Preventing Agents Going Off the Rails

Agents with real tools can cause real harm. A coding agent that executes arbitrary shell commands, a customer service agent with database write access, or a research agent with email capabilities all have the potential for serious damage if they go off script.

Guardrail categories:

1. **Input guardrails:** Filter or reject harmful, off-topic, or malicious user inputs before the agent processes them.
2. **Output guardrails:** Check agent responses for harmful content, data leaks, or policy violations before sending them to users.
3. **Action guardrails:** Restrict which tools the agent can call, with what parameters, and in what contexts.
4. **Scope guardrails:** Prevent the agent from taking actions outside its intended domain.

```

# Comprehensive guardrail system

class GuardrailSystem:
    def __init__(self, config: dict):
        self.rules = config.get("rules", [])
        self.blocked_patterns = config.get("blocked_patterns", [])
        self.allowed_tools = config.get("allowed_tools", [])
        self.tool_restrictions = config.get("tool_restrictions", {})
        self.max_cost_per_request = config.get("max_cost_per_request", 1.0)
        self.max_steps = config.get("max_steps", 25)

    def check_input(self, user_input: str) -> dict:
        """Check user input for safety issues."""
        issues = []

        # Check for prompt injection attempts
        injection_patterns = [
            "ignore previous instructions",
            "disregard your system prompt",
            "you are now",
            "new instructions:",
        ]

```

```

for pattern in injection_patterns:
    if pattern.lower() in user_input.lower():
        issues.append(f"Potential prompt injection: '{pattern}'")

# Check for blocked content patterns
for pattern in self.blocked_patterns:
    if re.search(pattern, user_input, re.IGNORECASE):
        issues.append(f"Blocked content pattern detected")

return {"safe": len(issues) == 0, "issues": issues}

def check_tool_call(self, tool_name: str, params: dict) -> dict:
    """Validate a tool call before execution."""
    issues = []

    # Check if tool is allowed
    if self.allowed_tools and tool_name not in self.allowed_tools:
        issues.append(f"Tool '{tool_name}' is not in the allowed list")

    # Check tool-specific restrictions
    restrictions = self.tool_restrictions.get(tool_name, {})

    if tool_name == "run_shell_command":
        # Block dangerous shell commands
        dangerous_commands = ["rm -rf", "sudo", "chmod 777", "curl | bash"]
        cmd = params.get("command", "")
        for dangerous in dangerous_commands:
            if dangerous in cmd:
                issues.append(f"Blocked dangerous command: '{dangerous}'")

    if tool_name == "query_database":
        # Block destructive SQL
        query = params.get("query", "").upper()
        if any(kw in query for kw in ["DROP", "TRUNCATE", "DELETE FROM"]):
            if not restrictions.get("allow_destructive", False):
                issues.append("Destructive SQL operations are not allowed")

    if tool_name == "send_email":
        # Restrict email recipients
        allowed_domains = restrictions.get("allowed_domains", [])
        recipient = params.get("to", "")
        domain = recipient.split("@")[-1] if "@" in recipient else ""
        if allowed_domains and domain not in allowed_domains:
            issues.append(f"Email to '{domain}' is not in allowed domains")

    return {"safe": len(issues) == 0, "issues": issues}

def check_output(self, output: str) -> dict:
    """Check agent output before sending to user."""
    issues = []

    # Check for PII leakage

```

```

pii_patterns = {
    "SSN": r"\b\d{3}-\d{2}-\d{4}\b",
    "Credit Card": r"\b\d{4}[\s-]?\d{4}[\s-]?\d{4}[\s-]?\d{4}\b",
    "Email": r"\b[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}\b",
}
for pii_type, pattern in pii_patterns.items():
    if re.search(pattern, output):
        issues.append(f"Potential {pii_type} in output")

return {"safe": len(issues) == 0, "issues": issues}

```

The principle of least privilege: Give agents only the permissions they need. A research agent should not have database write access. A customer service agent should not be able to execute arbitrary code. Restrict tool access based on the agent's role and the current task.

Cost Management: Agents Can Be Expensive

Agents are inherently more expensive than simple LLM calls because they make multiple calls per task. A 15-step agent using GPT-4 or Claude Opus can easily cost \$0.50-\$2.00 per run. At scale, this adds up fast.

Cost optimization strategies:

```

# Cost management implementation

class CostManager:
    def __init__(self, config: dict):
        # Cost per 1K tokens (example prices)
        self.model_costs = {
            "claude-opus-4-20250514": {"input": 0.015, "output": 0.075},
            "claude-sonnet-4-20250514": {"input": 0.003, "output": 0.015},
            "claude-haiku-3": {"input": 0.00025, "output": 0.00125},
        }
        self.budget_per_request = config.get("budget_per_request", 1.0)
        self.budget_per_day = config.get("budget_per_day", 100.0)
        self.daily_spend = 0.0

    def estimate_cost(self, model: str, input_tokens: int, output_tokens: int) -> float:
        costs = self.model_costs.get(model, {"input": 0.01, "output": 0.03})
        return (input_tokens / 1000 * costs["input"] +
                output_tokens / 1000 * costs["output"])

    def select_model_for_step(self, step_complexity: str) -> str:
        """Use cheaper models for simple steps, expensive models for complex ones."""
        if step_complexity == "simple":
            return "claude-haiku-3" # Cheap and fast
        elif step_complexity == "moderate":
            return "claude-sonnet-4-20250514" # Good balance
        else:
            return "claude-opus-4-20250514" # Best reasoning

```

```

def check_budget(self, estimated_cost: float) -> bool:
    """Check if we're within budget."""
    if self.daily_spend + estimated_cost > self.budget_per_day:
        raise CostLimitExceeded("Daily budget exceeded")
    if estimated_cost > self.budget_per_request:
        raise CostLimitExceeded("Per-request budget exceeded")
    return True

# Strategy 1: Model routing based on step complexity
# Strategy 2: Caching tool results to avoid redundant calls
# Strategy 3: Early termination when the answer is good enough
# Strategy 4: Prompt optimization to reduce token usage
# Strategy 5: Batch similar requests together

```

Key cost reduction techniques:

1. **Model routing.** Use the cheapest model that can handle each step. Simple tool calls and data extraction can use Haiku. Complex reasoning needs Opus. The savings from routing can be 70-80%.
2. **Caching.** Cache tool results (especially web searches and database queries) so repeated queries do not incur additional cost. A simple TTL cache can save 20-40% on tool-heavy agents.
3. **Prompt optimization.** Shorter system prompts, compressed tool descriptions, and summarized context all reduce token consumption. Every token counts when multiplied across thousands of agent runs.
4. **Early termination.** If the agent has a good-enough answer after 5 steps, stop. Do not let it run all 20 steps just because the limit allows it.
5. **Step limits.** Set reasonable maximum step counts based on empirical analysis of your agent's typical behavior.

Testing Agents: Unit Tests, Integration Tests, Evaluation Frameworks

Testing agents is fundamentally different from testing traditional software because agent behavior is non-deterministic. The same input can produce different outputs on different runs. This requires a multi-layered testing strategy.

```

# Agent testing strategy

# Layer 1: Unit tests for individual tools
class TestTools:
    def test_web_search_returns_results(self):
        result = web_search("Python programming language")
        assert result["success"] is True
        assert len(result["results"]) > 0

    def test_web_search_handles_empty_query(self):
        result = web_search("")
        assert result["success"] is False
        assert "error" in result

    def test_calculator_basic_arithmetic(self):
        result = calculate("2 + 2")
        assert result["result"] == 4

```



```

def test_calculator_handles_division_by_zero(self):
    result = calculate("1 / 0")
    assert result["success"] is False

# Layer 2: Integration tests for the agent loop
class TestAgentLoop:
    def test_agent_can_use_search_tool(self):
        """Agent should call web_search when asked about current events."""
        agent = create_test_agent()
        result = agent.run("What is today's weather in London?")

        # Verify the agent used the search tool
        assert any(step.tool_name == "web_search" for step in agent.execution_log)
        # Verify the result mentions weather
        assert "weather" in result.lower() or "temperature" in result.lower()

    def test_agent_terminates_within_step_limit(self):
        """Agent should not exceed the maximum step count."""
        agent = create_test_agent(max_steps=10)
        result = agent.run("Research and write a comprehensive report on quantum computing")
        assert len(agent.execution_log) <= 10

# Layer 3: Evaluation framework for quality
class AgentEvaluator:
    def __init__(self, test_cases: list):
        self.test_cases = test_cases
        # Each test case: {"input": str, "expected_criteria": list}

    def evaluate(self, agent) -> dict:
        results = []
        for case in self.test_cases:
            output = agent.run(case["input"])

            score = self.score_output(output, case["expected_criteria"])
            results.append({
                "input": case["input"],
                "output": output[:500],
                "score": score,
                "criteria_met": score["criteria_met"],
                "criteria_total": len(case["expected_criteria"])
            })

        avg_score = sum(r["score"]["overall"] for r in results) / len(results)
        return {"average_score": avg_score, "detailed_results": results}

    def score_output(self, output: str, criteria: list) -> dict:
        """Use an LLM to evaluate output against criteria."""
        evaluation = judge_llm.generate(
            f"Evaluate this output against the following criteria:\n"
            f"Output: {output}\n"

```

```

        f"Criteria: {json.dumps(criteria)}\n\n"
        f"For each criterion, score 0 (not met) or 1 (met).\n"
        f"Also provide an overall quality score from 0.0 to 1.0."
    )
    return parse_evaluation_scores(evaluation)

# Example test cases
test_cases = [
    {
        "input": "What are the three largest countries by area?",
        "expected_criteria": [
            "Mentions Russia as the largest",
            "Mentions Canada as second largest",
            "Mentions the United States or China as third",
            "Provides area figures",
            "Uses web search to verify current data"
        ]
    },
    {
        "input": "Calculate the compound interest on $10,000 at 5% for 10 years",
        "expected_criteria": [
            "Uses the calculator tool",
            "Provides the correct answer (approximately $16,288.95)",
            "Shows the formula used",
            "Explains the calculation"
        ]
    }
]

```

Human-in-the-Loop: When Agents Should Ask for Help

Not every task should be fully autonomous. **Human-in-the-loop (HITL)** patterns let agents escalate to humans when they encounter situations they cannot handle reliably.

```

# Human-in-the-loop implementation

class HITLAgent:
    def __init__(self, llm, tools, hitl_config: dict):
        self.llm = llm
        self.tools = tools
        self.confidence_threshold = hitl_config.get("confidence_threshold", 0.7)
        self.escalation_rules = hitl_config.get("escalation_rules", [])
        self.approval_required_tools = hitl_config.get("approval_required_tools",
        [])

    async def run(self, task: str) -> str:
        messages = [{"role": "user", "content": task}]

        for step in range(20):
            response = self.llm.generate(messages=messages, tools=self.tools)

```

```

if response.has_tool_call():
    tool_name = response.tool_call.name

    # Check if this tool requires human approval
    if tool_name in self.approval_required_tools:
        approved = await self.request_human_approval(
            f"Agent wants to use tool: {tool_name}\n"
            f"Parameters: {response.tool_call.arguments}\n"
            f"Context: {messages[-1]}"
        )
        if not approved:
            messages.append({
                "role": "system",
                "content": "Human denied this tool use. Try an
alternative approach."
            })
            continue

        result = self.execute_tool(response.tool_call)
        messages.append(tool_result_message(response, result))

    else:
        # Check confidence before returning
        confidence = self.assess_confidence(task, response.text, messages)

        if confidence < self.confidence_threshold:
            human_input = await self.escalate_to_human(
                f"Agent is not confident in its response.\n"
                f"Task: {task}\n"
                f"Proposed response: {response.text}\n"
                f"Confidence: {confidence}\n\n"
                f"Please review and either approve, edit, or provide
guidance."
            )

            if human_input["action"] == "approve":
                return response.text
            elif human_input["action"] == "edit":
                return human_input["edited_response"]
            elif human_input["action"] == "guidance":
                messages.append({
                    "role": "user",
                    "content": f"Additional guidance:
{human_input['guidance']}"
                })
                continue

            return response.text

def assess_confidence(self, task, response, messages) -> float:
    """Have the LLM self-assess its confidence."""
    assessment = self.llm.generate(

```

```

        f"Rate your confidence in this response from 0.0 to 1.0:\n"
        f"Task: {task}\nResponse: {response}\n"
        f"Consider: factual accuracy, completeness, and certainty."
    )
    return float(extract_number(assessment))

```

When agents should escalate:

- Confidence is low on a high-stakes decision
- The task involves irreversible actions (sending emails, modifying production data)
- The agent encounters a situation not covered by its instructions
- Multiple tool failures suggest the task is beyond the agent's capability
- The user's request is ambiguous and making assumptions could lead to the wrong result

Deployment Patterns and Infrastructure

Deploying agents requires infrastructure that handles their unique characteristics: variable execution time, multiple external calls per request, and potentially high resource consumption.

Deployment architecture

```
class AgentDeployment:
```

```
    """
```

```
    Architecture:
```

```

    [API Gateway]  ->  [Request Queue]  ->  [Agent Workers]
      |                |                  |
    [Rate Limiter]  [Redis/SQS]         [Runs agent loop]
      |                |                  |
    [Auth/RBAC]                [Tool APIs, LLM APIs]
                              |
                              [Result Store]
                              |
                              [Response to User]

```

```
    Key design decisions:
```

- ```

 - Async execution: agent runs are long-running, use background workers
 - Streaming: return partial results as the agent works
 - Caching: cache tool results and common queries
 - Circuit breakers: prevent cascade failures from external APIs

```

```
 """
```

```
 def __init__(self, config):
```

```
 self.queue = MessageQueue(config.queue_url)
```

```
 self.cache = Cache(config.redis_url, ttl=3600)
```

```

 self.worker_pool = WorkerPool(
 min_workers=config.min_workers,
 max_workers=config.max_workers,
 autoscale=True
)

```

```
 async def handle_request(self, request: AgentRequest) -> str:
```

```

Check cache first
cache_key = self.compute_cache_key(request)
cached = await self.cache.get(cache_key)
if cached:
 return cached

Enqueue for processing
job_id = await self.queue.enqueue(request)

Return job ID for async polling (or use WebSocket for streaming)
return {"job_id": job_id, "status": "processing"}

async def process_job(self, job: AgentJob):
 """Worker process that runs the agent."""
 agent = ProductionAgent(job.config)

 try:
 result = await agent.run(job.request)
 await self.cache.set(job.cache_key, result, ttl=3600)
 await self.notify_completion(job.id, result)
 except Exception as e:
 await self.notify_failure(job.id, str(e))

```

## Case Study: Full Agent from Concept to Production

Let us trace the full lifecycle of building a production agent: a **customer support agent** for a SaaS product.

**Phase 1: Define the scope.** The agent should handle common customer inquiries: account questions, billing issues, feature questions, and basic troubleshooting. It should escalate to humans for complex issues, refunds over \$100, and account cancellations.

**Phase 2: Design the tools.**

```

Customer support agent tools
support_tools = [
 Tool("lookup_customer", "Look up customer by email or ID", customer_db_lookup),
 Tool("check_subscription", "Check customer subscription status and history",
sub_check),
 Tool("search_knowledge_base", "Search product documentation and FAQ",
kb_search),
 Tool("check_known_issues", "Check for known bugs and outages", issue_tracker),
 Tool("create_ticket", "Create a support ticket for human follow-up",
ticket_create),
 Tool("apply_credit", "Apply account credit (max $50)", credit_apply),
 Tool("reset_password", "Send password reset email to customer", password_reset),
]

```

**Phase 3: Build the prototype.** Implement the basic agent loop with the tools, a system prompt defining the agent's behavior, and guardrails for the sensitive operations.

**Phase 4: Test extensively.** Create 100+ test cases covering common scenarios, edge cases, and adversarial inputs. Evaluate using both automated criteria and human review. Iterate on the system prompt and tool

descriptions until quality meets the threshold.

**Phase 5: Add production infrastructure.** Logging, tracing, cost tracking, HITL escalation, guardrails, caching, and error handling. This typically doubles the codebase.

**Phase 6: Gradual rollout.** Start with 5% of customer inquiries routed to the agent, with every response reviewed by a human. Increase to 25%, then 50%, then full deployment as confidence grows. Maintain a human review sample (5%) indefinitely.

```
Production customer support agent (simplified)

class CustomerSupportAgent:
 def __init__(self):
 self.agent = ProductionAgent(
 model="claude-sonnet-4-20250514",
 tools=support_tools,
 system_prompt=SUPPORT_SYSTEM_PROMPT,
 guardrails=GuardrailSystem({
 "max_steps": 10,
 "max_cost_per_request": 0.50,
 "blocked_patterns": [r"ignore.*instructions", r"system.*prompt"],
 "tool_restrictions": {
 "apply_credit": {"max_amount": 50},
 "create_ticket": {"require_customer_lookup_first": True}
 },
 "escalation_triggers": [
 "refund over $100",
 "account cancellation",
 "legal threat",
 "data deletion request"
]
 }),
 hitl_config={
 "approval_required_tools": ["apply_credit"],
 "confidence_threshold": 0.75,
 "escalation_callback": notify_support_team
 }
)
 self.logger = AgentLogger("customer_support")
 self.evaluator = ResponseEvaluator()

 async def handle_inquiry(self, customer_email: str, message: str) -> dict:
 # Log the incoming inquiry
 self.logger.log("inquiry_received", customer_email=customer_email)

 # Build context with customer info
 context = f"Customer email: {customer_email}\nInquiry: {message}"

 # Run the agent
 response = await self.agent.run(context)

 # Evaluate response quality
```

```

quality = self.evaluator.quick_check(response)

if quality["score"] < 0.7:
 # Low quality -- escalate to human
 self.logger.log("low_quality_escalation", score=quality["score"])
 ticket = await create_escalation_ticket(customer_email, message,
response)
 return {
 "response": "I want to make sure you get the best help possible. "
 "I've escalated your inquiry to our support team. "
 "You'll hear back within 2 hours.",
 "escalated": True,
 "ticket_id": ticket.id
 }

 return {
 "response": response,
 "escalated": False,
 "trace_id": self.agent.last_trace_id
 }

```

**Phase 7: Monitor and iterate.** Track resolution rates, customer satisfaction scores, escalation rates, and cost per resolution. Use the data to continuously improve the system prompt, add new tools, and refine guardrails.

## Key Takeaways

- The gap between **prototype and production** is enormous. Plan for reliability, scale, observability, cost, and safety from the start.
- **Observability** (structured logging, distributed tracing, metrics dashboards) is not optional. You cannot operate what you cannot observe.
- **Safety guardrails** must cover inputs, outputs, and actions. Apply the principle of least privilege to tool access.
- **Cost management** is critical at scale. Use model routing, caching, prompt optimization, and early termination to control expenses.
- **Testing agents** requires a multi-layered approach: unit tests for tools, integration tests for the loop, and LLM-based evaluation for output quality.
- **Human-in-the-loop** patterns let agents escalate gracefully when they are uncertain or when tasks are high-stakes.
- **Deploy gradually** with human oversight. Start with a small percentage of traffic and increase as confidence in the system grows.
- Production agents are **living systems**. Plan for continuous monitoring, evaluation, and improvement after deployment.